



**האוניברסיטה הפתוחה
המחלקה למתמטיקה ולמדעי המחשב
החטיבה למדעי המחשב**

פרויקט מתקדם במדעי המחשב

מימוש סוכן AI אוטונומי לניהול אישורי הגעה (RSVP) מבוסס מודלי שפה גדולים (LLM) ושפה טבעית

מנחה: פרופ' אהוד גודס

מגיש: ישראל הייבלום

סמסטר: 2026 ב'

גרסה 2

פרק 1: מבוא	4
1.1 רקע - עולם ה-Conversational AI	4
1.2 האתגר והבעיה	4
1.3 מטרת הפרויקט ותרומתו	4
פרק 2: רקע מדעי וסקירת ספרות	5
2.1 עיבוד שפה טבעית בעברית - שפה "דלה במשאבים" בסביבה עשירה בתוכן	5
2.2 מודלים טרנספורמטיביים ולמידה מהקשר (In-Context Learning)	5
2.3 מעבר פרדיגמטי: מ FSM לסוכנים מבוססי גרף	6
2.4 Neuro-Symbolic AI מדוע לא רק LLM?	6
פרק 3: ניתוח הבעיה והגדרת דרישות	8
3.1 מורכבות השיח ופענוח כוונות	8
3.2 ניהול מצב והקשר דינמי	9
3.3 הגדרת דרישות פונקציונליות וארכיטקטוניות	9
פרק 4: ארכיטקטורת המערכת	10
4.1 מבנה כללי Monorepo ושלוש שכבות	11
4.2 ארכיטקטורה משושה Ports & Adapters	11
4.3 מנוע ניהול השיחה LangGraph - כגרף מצבים	12
4.4 EffectsPatch - עדכון דליל ומדויק	12
פרק 5: אלגוריתמיקה ומימוש הסוכן	15
5.1 נירמול טקסט עברי - לפני הכל	15
5.3 חילוץ כמות - שרשרת עדיפויות בשלב 14	15
5.4 Levenshtein Fuzzy Matching	16
5.5 הפעלת מודל השפה - Prompt Engineering	16
5.6 טיפול בתיקונים ושינויי כוונה	17
פרק 6: ממשק המשתמש והניהול (Admin Web)	18
6.1 ארכיטקטורת צד הלקוח	18
6.2 מסכי הממשק	19
6.3 מנגנון הזיהוי האישי- Telegram Deep Linking	21
6.4 לוקליזציה לשפה ו-UX	21
פרק 7: ניתוח תוצאות והערכה	22
7.1 מתודולוגיית ההערכה	22
7.2 הערכה כמותית - F1-F Confusion Matrix, Precision, Recall	23
7.3 ניתוח איכותני של שיחות רב-שלביות מתוך יומני המערכת (Logs)	27
7.4 הערכת חילוץ ישויות כמות (Headcount Extraction)	29

29.....	7.5 יעילות חישובית וחלוקת עומס מערכתי (Load Distribution)
30.....	7.6 התפלגות ציוני ודאות (Confidence Distribution)
30.....	7.7 דיון: מגבלות מתודולוגיות של הליך ההערכה
31.....	7.8 סיכום ממצאי ההערכה
33	פרק 8: מבנה תוכנה SDLC בעידן ה-AI וה-LLM
33.....	8.1 תהליך התכנון
33.....	8.2 מסגרת מתודולוגית: פיתוח אינקרמנטלי בסביבת מפתח יחיד
33.....	8.3 אבולוציית הפיתוח - ארבעה גלים
34.....	8.4 כלי AI בתהליך הפיתוח (Coding Assistance)
34.....	8.5 שימוש ב LLM-כחלק מריצת המערכת (Runtime AI)
34.....	8.6 ניהול ידע והנחיית סוכנים (Agentic Instruction Layers)
35.....	8.7 הנדסת הקשר (Context Engineering) בתהליך הפיתוח
36	פרק 9: דיון וסיכום
36.....	9.1 סיכום ההישגים
37.....	9.2 שלוש תובנות ארכיטקטוניות שנצרכו
37.....	9.3 שינוי פרדיגמה: מהנדס תוכנה בעידן ה-AI
37.....	9.4 כיווני המשך ומחקר עתידי
39	פרק 10: ביבליוגרפיה
40	נספח: סכמת בסיס הנתונים
40.....	1.1 סקירת מודל הנתונים ועקרונות התכנון
40.....	1.2 Collections ותיאור השדות

1.1 רקע - עולם ה-Conversational AI

בעשור האחרון חל שינוי מהותי באופן שבו מערכות תוכנה מתקשרות עם בני אדם. ממשקי משתמש מבוססי שיחה (Conversational User Interfaces - CUI) עברו מעצי החלטה נוקשים ומוגבלים, שבהם המשתמש היה נאלץ לבחור מתפריט מוגדר מראש, אל מערכות שמסוגלות להבין שפה חופשית ולנהל דיאלוג טבעי. המעבר הזה לא היה הדרגתי אלא הוא התרחש בקפיצה, עם הופעתם של מודלי שפה גדולים (LLMs-Large Language Models) כדוגמת GPT וקלוד, שהפכו הבנת שפה טבעית לנגישה ומעשית גם מחוץ למעבדות מחקר. בהקשר זה, ניהול אישורי הגעה לאירועים (RSVP) הוא דוגמה מעניינת במיוחד: מבחוץ היא נראית כמשימה פשוטה של לאסוף "כן" או "לא" ממספר אנשים. אך בפועל, היא מציפה מספר אתגרי הבנה ותיאום שאדם מטפל בהם בקלות אך מכונה מתקשה איתם: אנשים עונים בסגנונות שונים, משתמשים בסלנג תרבותי, לפעמים עונים בעמימות מכוונת, ולעתים מתחרטים ומתקנים את עצמם. בניית מערכת שתמודד עם כל אלה בשפה העברית, בשיחה חופשית, ללא טפסים, היא בדיוק הבעיה שפרויקט זה ניגש לפתור.

1.2 האתגר והבעיה

מערכות RSVP מסורתיות מתבססות על טפסים מקוונים או שאלות בינאריות. הבעיה המרכזית עם גישה זו היא ה"חיכוך" (Friction) שהיא יוצרת: המוזמן מקבל הודעה בוואטסאפ, אבל כדי לאשר הגעה הוא חייב לפתוח דפדפן, ללחוץ על קישור, להכנס לאתר ולמלא טופס. מחקרי UX מראים שכל שלב נוסף מקטין משמעותית את שיעורי ההשלמה. מנגד, לאפשר לאנשים להשיב חופשית בהודעת טקסט פירושו שמישהו צריך לקרוא ולסדר מאות תשובות. עומס שהופך לבעיה רצינית בכל אירוע בינוני ומעלה.

הבעיה המחקרית שפרויקט זה מטפל בה ניתן לנסח כך:

האם ניתן לבנות סוכן אוטונומי שיפרש תשובות RSVP בטקסט חופשי בשפה העברית, יסיק כוונות מדויקות, ינהל שיחה רב-שלבית עד להשלמת המידע הנדרש כאשר כל זאת ללא כל התערבות אנושית?

האתגר מורכב משלושה ממדים עיקריים:

ממד הבנת השפה: עברית מציבה קשיים ייחודיים כמו היעדר ניקוד יוצר עמימות מורפולוגית, ביטויים כמו "בלי נדר" או "נשתדל" דורשים הבנה תרבותית לא מפורשת, ושגיאות כתיב נפוצות ("שניפ" במקום "שניים") צריכות להיות מטופלות בסלחנות ולא כשגיאות.

ממד ניהול השיחה: השיחה אינה בהכרח חד-שלבית. מוזמן יכול לענות "בטח", ואז הסוכן צריך לשאול לגבי מספר המגיעים, ואז המוזמן עשוי לתקן את תשובתו. המערכת צריכה לשמור הקשר, לזהות תיקונים, ולסיים את השיחה רק כשכל המידע הנדרש הושלם.

ממד האמינות והתחזוקה: מערכת המסתמכת רק על LLM היא הסתברותית ויקרה להפעלה. מערכת המסתמכת רק על חוקים שבירה ולא מסוגלת להתמודד עם מקרי קצה. הפרויקט חקר כיצד לשלב בין שתי הגישות.

1.3 מטרת הפרויקט ותרומתו

מטרת העבודה היא תכנון, פיתוח והדגמה של מערכת RSVP מקצה לקצה (**End-to-End**), המונעת על ידי סוכן AI אוטונומי. הפרויקט, המכונה **EZ-Event-BOT**, נועד לגשר על הפער בין פשטות הרישום בטפסים קשיחים לבין הגמישות של שיח אנושי טבעי.

יעדי המערכת המרכזיים:

- **ממשק שיח אישי:** מימוש בוט על פלטפורמת **Telegram** אליו מגיעים המוזמנים דרך קישור ייחודי המספק הקשר אישי לשיחה.
- **פענוח שפה חופשית:** ניהול דיאלוג בעברית טבעית, תוך שימוש ביכולות **NLU** מתקדמות לפירוש כוונות, חילוף נתונים (כמות משתתפים) וטיפול בעמימות.

- **ניהול ושקיפות:** עדכון המארח בזמן אמת על סטטוס ההגעה דרך ממשק ניהול ויזואלי (**Admin Dashboard**) מבוסס WEB.
- **תיעוד הנדסי:** פרטי המימוש, הארכיטקטורה והלוגיקה מתועדים במלואם במאגר קוד פתוח, המנגיש את הידע שנצבר לקהילת הפיתוח.

התרומה המדעית-הנדסית של הפרויקט מתמקדת בשלושה תחומים מרכזיים:

- (א) **עיצוב צינור NLU היברידי לעברית:** שילוב מנוע חוקים דטרמיניסטי עם מודל שפה גדול (LLM) כ-**Fallback**. גישה זו מאפשרת טיפול יעיל בכ-80% מהמקרים ללא צורך בקריאות API חיצוניות, תוך שימור היכולת לפרש ביטויים תרבותיים ועמומים שמנוע חוקים לבדו אינו מסוגל לפענח.
- (ב) **ארכיטקטורת סוכן מבוססת גרף מצבים (LangGraph):** הטמעת תשתית המאפשרת ניהול שיח רב-שלבי המגיב בזמן אמת לתיקונים ושינויי כוונה מצד המשתמש. פתרון זה מציג אלטרנטיבה מודרנית לגישות ה-**FSM** (מכונת מצבים סופית) המסורתיות, הסובלות מבעיית "פיצוץ מצבים" (**State Explosion**) וחוסר גמישות מול שפה חופשית.
- (ג) **יישום ארכיטקטורה משושה (Hexagonal Architecture) בסוכני AI:** הדגמה מעשית של הפרדה מוחלטת בין לוגיקת השיח (הדומיין) לבין התשתית הטכנולוגית. מבנה זה מבטיח מודולריות גבוהה ומאפשר החלפת ספק מודל השפה או פלטפורמת המסרים ללא צורך בשינוי של הלוגיקה העסקית של הסוכן.

פרק 2: רקע מדעי וסקירת ספרות

2.1 עיבוד שפה טבעית בעברית - שפה "דלה במשאבים" בסביבה עשירה בתוכן

- כדי להבין את הבחירות הטכנולוגיות שנעשו בפרויקט, חשוב להתחיל בנקודת המוצא: העברית היא שפה מאתגרת במיוחד לעיבוד אוטומטי. בניגוד לאנגלית, שעבורה פותחו מאות מודלים, כלים ובנצ'מרקים, העברית נחשבת "שפה דלה במשאבים" (Low-Resource Language). כלומר, כמות הנתונים המתויגת הזמינה לאימון מודלים היא מצומצמת יחסית.
- אך הבעיה עמוקה יותר מאשר סתם מחסור בנתונים. העברית מציבה שלושה אתגרים מבניים ייחודיים שכל מערכת NLP חייבת להתמודד איתם:
- **מורפולוגיה עשירה ואגלוטינטיבית.** מילה עברית אחת יכולה לקפל בתוכה מידע שבאנגלית דורש מספר מילים. "שהולכים" מכיל יחד: שייכות (ש-), פעולה (הולך), רבים (-ים) ותחילויות נוספות. בהקשר של RSVP, הטיות כמו "מגיעים", "נגיע", "יגיעו" ו"אגיע" כולן מבטאות כוונת הגעה אך שונות מורפולוגית לגמרי.
 - **עמימות ניקוד.** בעברית מודרנית ללא ניקוד, מילים רבות זהות בכתיבתן אך שונות במשמעותן. "מגיע" יכול להיות "הוא מגיע" (הגעה) או "מגיע לו" (מגיע כמו ראוי). כלי NLP הפועל ללא ניקוד חייב לסמוך על ההקשר.
 - **ביטויים תרבותיים בלתי-פורמליים.** זוהי אולי האתגר המעניין ביותר: ביטויים כמו "בלי נדר", "אם יהיה לי כוח" או "נשתדל" הם תשובות ישראליות לגיטימיות שדורשות הבנה סוציו-תרבותית ולא רק לשונית. בהקשר של RSVP, כל אלו הם וריאציות של MAYBE, אך אף מילה בהן אינה "אולי".

כדי להתמודד עם מורפולוגיה עשירה, הפרויקט מממש נירמול טקסט עברי (Hebrew Text Normalization) לפני כל ניתוח: הסרת ניקוד, קיצוץ אותיות שימוש קדם-מילתיות (ו, ה, ב, ל, כ, מ, ש), וטוקניזציה. גישה זו נפוצה בספרות ונחשבת אבן יסוד לעיבוד עברית (Seker et al., 2021). מודלים עבריים מודרניים כגון AlephBERT (Seker et al., 2021) מדגימים את האתגרים הייחודיים של עברית כשפה דלת-משאבים ואת הצורך בנירמול טקסט כשלב מקדים. המטרה להביא למודל data איכותי ככל שניתן.

2.2 מודלים טרנספורמטיביים ולמידה מהקשר (In-Context Learning)

התפתחות מודלי השפה כבסיס לסוכן ה-RSVP:

- **ארכיטקטורת האם (Transformer, 2017):** המעבר למנגנוני תשומת לב (Attention).

- **שלב ההבנה (BERT, 2018):** דגש על הבנת הקשר משפטי דו-כיווני (Devlin et al., 2019).
- **שלב היצירתיות וההקשר (GPT-3, 2020):** הצגת יכולות ה-Few-Shot.
- **שלב הסוכנות וההסקה (+GPT-4 / Claude, 2023):** מודלים המסוגלים לבצע הנחיות מורכבות ולנהל שיח אוטונומי.

האתגר והפתרון הטכנולוגי

מה שחסר בכל הגישות הללו:

- **BERT:** מצטיין בהבנה, אך אינו מסוגל לנהל שיח רב-תורות.
- **GPT-3 ו GPT-4:** מסוגלים לנהל שיח, אך בהיעדר ניהול מצב מפורש - (**State Management**) כל פנייה נחשבת עצמאית והמידע אינו מצטבר בצורה מובנית.

פרויקט זה מתמקם בצומת שבין שלושת הצרכים:

1. **הבנה מורפולוגית: (NLU)** זיהוי מדויק של כוונת המשתמש.
 2. **ניהול מצב מפורש: (LangGraph)** שמירה על רצף לוגי לאורך זמן.
 3. **אוטונומיה בקבלת החלטות** שימוש בפונקציה `decideAction`, לצורך הכרעה על הצעד הבא בשיחה.
- הנקודה הקריטית לפרויקט זה היא שמודלים אלו אינם דורשים Fine-Tuning על דאטה-סט מתויג לשפה העברית. במקום זאת, הם מנצלים **In-Context Learning** - היכולת ללמוד את המשימה מדוגמאות שמזרקות ישירות לתוך ה-Prompt. בפרויקט, זה מומש כך: ה-System Prompt של ה-LLM כולל חמש דוגמאות מייצגות של קלט/פלט בעברית, שמלמדות את המודל כיצד לסווג ביטויים שאינם מכוסים על ידי מנוע החוקים, כגון "אני ואשתי ו-2 ילדים" ($\rightarrow$ YES, headcount: 4) או "כן, אבל תלוי בסיפורים" ($\rightarrow$ MAYBE). הדגש החשוב הוא שגישת Few-Shot מאפשרת לנו להתאים מודל כללי לדומיין ספציפי מאוד (RSVP בעברית) ללא עלות אימון, וזה בדיוק מה שהופך אותה לרלוונטית לפרויקט.

2.3 מעבר פרידיגמטי: מ FSM לסוכנים מבוססי גרף

- כדי להבין מדוע הפרויקט בחר ב-LangGraph לניהול השיחה, כדאי להבין את הרצף ההיסטורי של גישות לניהול דיאלוג:
- **דור ראשון - עצי החלטה (Decision Trees):** מערכות RSVP מסורתיות משתמשות בלוגיקה if-else פשוטה: "אם הודעה == 'כן' $\rightarrow$ עדכן YES". גישה זו דטרמיניסטית ואמינה, אך שבירה לחלוטין מול כל קלט שלא תוכנן מראש.
 - **דור שני - מכונות מצבים סופיות (FSM - Finite State Machine):** שיפור על עצי החלטה - המערכת מנהלת "מצב נוכחי" ומאפשרת מעברים בין מצבים. הבעיה: ה-FSM סובל מ"פיצוץ מצבים" (State Explosion). ככל שמוסיפים מקרי קצה (תיקון, שינוי דעה, שאלות הבהרה), מספר המצבים הנדרש גדל בצורה בלתי ניהולית.
 - **דור שלישי - סוכנים מבוססי-LLM (Pure LLM Agents):** מסירים את כל המגבלות הקשיחות ומשתמשים במודל שפה לכל החלטה. גמיש ביותר, אך יקר, איטי ולא דטרמיניסטי. לכן בעייתי כשנדרשת אמינות גבוהה.
- הגישה הנוכחית - גרפי מצבים (State Graphs):** המסגרת (LangChain, 2023) LangGraph מציעה פשרה אלגנטית. במקום לבחור בין קשיחות לגמישות, היא מאפשרת להגדיר גרף שבו כל צומת הוא פונקציה טהורה שפועלת על "מצב משותף" (Shared State), וניתוב בין הצמתים מותנה בתוכן המצב. זו הגישה שיושמה בפרויקט - ונסביר אותה בפירוט בפרק הרביעי.

2.4 Neuro-Symbolic AI מדוע לא רק? LLM

ספרות ה-AI העדכנית מצביעה על מתח מרכזי בבניית מערכות מעשיות: מצד אחד, מודלי שפה גדולים (LLMs) מסוגלים להתמודד עם כמעט כל קלט בשפה טבעית. מצד שני, הם מאופיינים בעלויות מחשוב גבוהות, זמני תגובה איטיים וחוסר דטרמיניזם.

LeCun (2022) הציע ארכיטקטורה חלופית לאינטליגנציה אוטונומית המבוססת על מודלי עולם Joint Embedding, בטענה שגנרטיביות לבדה (כפי שקיימת ב-LLMs) אינה מספקת להבנה אמיתית של העולם. במקביל, Marcus (2020) דחף לאימוץ גישה היברידית המשלבת לוגיקה סמלית (Symbolic AI) עם למידת מכונה, בטענה שרשתות נוירוניות טהורות מצטיינות בהכללה בתוך תחום ההכשרה, אך נכשלות מחוצה לו.

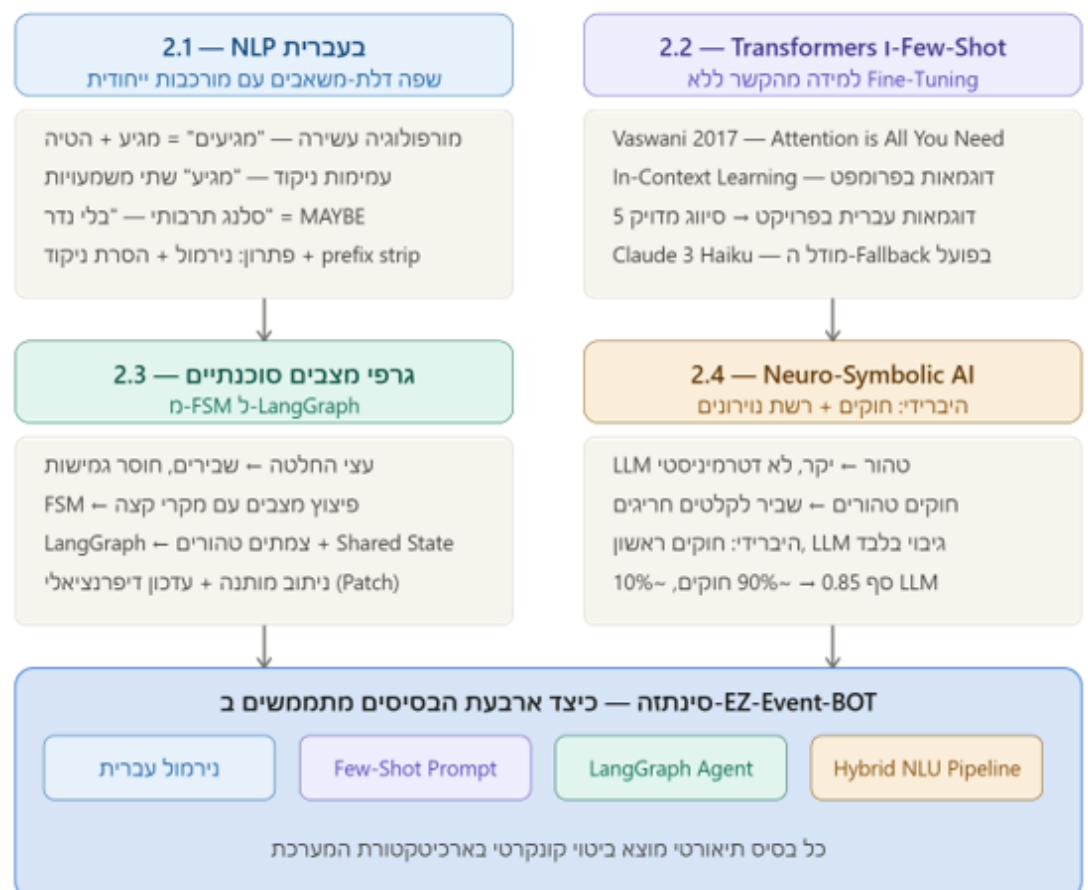
שני החוקרים, למרות נקודות מוצא שונות, תומכים בנחיצותו של הכיוון שנבחר בפרויקט זה: שילוב בין מנוע חוקים דטרמיניסטי (Symbolic/Rule-based) לטיפול במצבי הליכה, לבין LLM המשמש כFallback גמיש. גישה זו, המכונה לעיתים Neuro-symbolic AI, מאפשרת ליהנות מהאמינות והמהירות של מערכות מבוססות חוקים, מבלי לוותר על יכולת ההבנה הרחבה של מודלים טרנספורמריים מודרניים.

הגישה ה-Neuro-Symbolic משלבת בין "לוגיקה סמלית" (Symbolic - חוקים ברורים, דטרמיניסטיים) לבין "רשתות נוירונים" (Neural - גמישות, הסתברותיות). בפרויקט זה, הגישה ממומשת בצורה מאוד קונקרטית:

- **הרובד הסמלי:** מנוע חוקים (~630 שורות קוד) שמבצע נירמול, התאמת מילות מפתח, ואלגוריתם מרחק לוינשטיין. הכל דטרמיניסטי ומהיר.
- **הרובד הניורוני:** Claude 3 Haiku מופעל אך ורק כ-Fallback כאשר ציון המהימנות של החוקים נופל מתחת לסף 0.85.

התוצאה, כפי שמתועדת בקוד ובתיעוד הפרויקט: כ-80%-85% מההודעות מטופלות על ידי מנוע החוקים בלבד, ורק הזנב הארוך (Long Tail) של הודעות מורכבות מגיע ל-LLM. רוב האורחים עונים תשובות ברורות של "כן", "לא" ומספר של נלווים, כ-20% מאלו שעונים להודעה, ישיבו טקסט עמום שידרוש בינה אנושית או לפחות מלאכותית לצורך פענוח.

איור 1 – סיכום פרק 2 – רקע מדעי: ארבעה עמודי תווך תיאורטיים



האיור ממחיש את הזיקה הישירה בין הבסיס התיאורטי לבין המימוש ההנדסי בפרויקט. המבנה ההיררכי מציג את המעבר מתיאוריה ליישום:

- **השכבה התיאורטית (ארבעת הריבועים העליונים):** מייצגת תחומי מחקר עצמאיים המהווים את עמודי התווך של העבודה - עיבוד שפה טבעית (NLP) בעברית, למידה מהקשר (**In-Context Learning**), ניהול דיאלוג מבוסס-גרף, וגישת ה-**Neuro-Symbolic AI**. תחומים אלו נסקרים בהרחבה בסעיפים 2.1-2.4 בפרק סקירת הספרות.
- **השכבה היישומית (השורה התחתונה):** מפרטת את ארבעת רכיבי המערכת שבהם עקרונות אלו מתורגמים לפרקטיקה הנדסית.

מבנה זה מוכיח כי כל הכרעה ארכיטקטונית בפרויקט אינה מקרית, אלא נסמכת על בסיס מחקרי מוצק המשלב יכולות הסקה של מודלי שפה גדולים עם חוסן של מערכות מבוססות לוגיקה וגרפים.

פרק 3: ניתוח הבעיה והגדרת דרישות

בניגוד לפרקי הרקע התיאורטי, פרק זה יורד לקרקע: אילו בעיות ספציפיות עלו בתכנון המערכת, ומה בדיוק נדרש מהסוכן כדי להתמודד איתן.

3.1 מורכבות השיח ופענוח כוונות

הבעיה הנראית פשוטה - "האם האורח מגיע?", מסתבכת מהרגע הראשון שנוגעים בה. מערכת RSVP לא מקבלת קלט מובנה, היא מקבלת טקסט חופשי ממוזמנים שמקלידים על הטלפון, לעיתים בעצלנות, לעיתים בהומור, ולעיתים בעמימות מכוונת. הפער בין השפה הטבעית לבין המבנה הלוגי שהמערכת צריכה ($rsvpStatus \in \{YES, NO, MAYBE\} + headcount$) הוא גדול ומלא מוקשים. ($\mathbb{Z}^+ \cup \{null\}$)

ניתן לסווג את אתגרי הפענוח לארבע קטגוריות עיקריות, כפי שמשקפות ישירות בנתוני הדמו ובתיעוד הפרויקט: **הסקה עקיפה**. ההודעה "אנחנו זוג" אינה מכילה את המילה "כן" ואינה מכילה מספר, אך דורשת מהסוכן להסיק בו-זמנית גם $rsvpStatus: YES$ וגם $headcount: 2$. הסוכן צריך לזהות שמילת היחס "זוג" היא הן אישור הגעה והן כמות. **מידע חלקי**. "בטח שמגיע!" הוא אישור חד-משמעי, אך הוא לא מספר כמה אנשים מגיעים. הסוכן לא יכול לסיים את השיחה; עליו לשאול שאלת המשך ולעבור למצב $YES_AWAITING_HEADCOUNT$. **עמימות וסייגים תרבותיים**. "תלוי בעבודה, עוד לא סגור" אינה מכילה אף אחת ממילות המפתח הקלאסיות של אי-ודאות. הזיהוי שמדובר ב-MAYBE מגיע דרך מילת המפתח "תלוי", שמשמעותה התרבותית ידועה אך אינה בהכרח מפורשת בכל מערכת NLP.

תיקונים ושינויי כוונה. זהו אולי האתגר הכי עדין: "אופס טעיתי, נהיה רק 2" מגיעה לאחר שהאורח כבר אישר הגעה של 4 איש. המילה "טעיתי" היא מילת תיקון (correction keyword), אך "נהיה רק 2" יכולה לפי ניתוח נאיבי להיפרש כ-NO עם כמות - לא כעדכון כמות בתוך YES. פונקציית $decideAction$ בקוד מטפלת בדיוק במקרה הזה:

```
decideAction.ts
טיפול בתיקון כמות מול שינוי סטטוס
if(
    guestContext.currentRsvpStatus === 'YES' &&
    hasCorrectionKeyword &&
    interpretation.headcountExtraction.kind === 'exact'
){
    עדיפות: עדכון כמות לא ביטול התשובה החיובית
    return { type: 'SET_RSVP', rsvpStatus: 'YES', headcount: ... };
}
```

הטבלה הבאה מרכזת את טווח הקלטים שהמערכת נדרשה לפרש ואת הסיווג הנכון:

הודעת אורח	rsvpStatus	headcount	אתגר עיקרי
"כן מגיע"	YES	null	מידע חלקי - יש לשאול
"אנחנו זוג"	YES	2	הסקה עקיפה

אתגר עיקרי	headcount	rsvpStatus	הודעת אורח
חיבור מרכיבים	4	YES	"אני ואשתי ו-2 ילדים"
סייג תרבותי	null	MAYBE	"תלוי בעבודה"
פשוט אך ברור	null	NO	"לא אוכל להגיע, סליחה"
תיקון כמות ≠ ביטול	2	YES	"אופס טעיתי, נהיה 2"
ACK_NO_CHANGE בלבד	—	—	"שלום" (אורח שכבר אישר)

3.2 ניהול מצב והקשר דינמי

עמוד התווך השני של הבעיה הוא ניהול מצב. שיחת RSVP אינה מתכנסת תמיד לתשובה מיידית, היא מתפתחת על פני מספר תורות (Turns). לכן, מערכת שיחה חסרת-מצב (Stateless) אינה אפשרית כאן. הסוכן מנהל בכל רגע נתון שני ממדי מצב מקבילים:

rsvpStatus: הסטטוס הלוגי של האורח - YES | NO | MAYBE | NO_RESPONSE. זהו המידע שמעניין את המארח.

conversationState: מצב השיח הנוכחי - YES_AWAITING_HEADCOUNT | DEFAULT. זהו המידע שמכוון את הסוכן לאיזה צומת בגרף לנתב את ההודעה הבאה.

השילוב של שני הממדים יוצר לוגיקת ניתוב עשירה. לדוגמה: אורח שנמצא במצב YES_AWAITING_HEADCOUNT ושולח "לא יכול בסוף", הסוכן צריך לזהות שמדובר בשינוי כוונה, לצאת מלולאת שאלת הכמות, ולעדכן את הסטטוס ל-NO. ההחלטה מקופלת בפונקציה הטהורה decideAction שבה מרוכזת כל לוגיקת העסקים של המערכת, ללא side-effects, עם 19 בדיקות יחידה.

נקודה נוספת שחייבת לבוא לידי ביטוי בתכנון: שינוי אמיתי בנתונים לעומת תגובה שאינה משנה כלום. אם אורח שכבר אישר הגעה של 2 איש שולח שוב "כן מגיע", הסוכן מזהה שאין שינוי (ACK_NO_CHANGE) ועונה בנימוס מבלי לכתוב לבסיס הנתונים. מנגנון זה נקרא ה-EffectsPatch הדליל, הוא אחד מהחלטות התכנון המשמעותיות ביותר בפרויקט, ונפרט עליו בפרק 4.

3.3 הגדרת דרישות פונקציונליות וארכיטקטוניות

לאחר ניתוח האתגרים, ניתן לנסח את דרישות המערכת באופן מדויק:

דרישה 1: אוטונומיה מלאה בניהול שיח. הסוכן חייב לקבל בכל תור שלושה סוגי החלטות ללא התערבות אנושית: (א) האם לעדכן נתונים בבסיס הנתונים (ב) האם לשאול שאלת הבהרה ואיזו (ג) האם להכיר בכך שהשיחה הסתיימה ולסגור אותה בנימוס.

לוגיקת ההחלטה כולה מגולמת ב-**decideAction**: פונקציה סינכרונית טהורה שמקבלת את המצב ומחזירה פעולה.

דרישה 2: גישה היברידית לפענוח (Hybrid NLU). על המערכת לנצל שיטות דטרמיניסטיות (חוקים וביטויים רגולריים) תחילה, ולהפעיל LLM רק כגיבוי. זה לא רק שיקול עלות אלא זה גם שיקול אמינות. תוצאת LLM שאינה עקבית עלולה לגרום לעדכוני שגויים בבסיס הנתונים. הסף שנבחר בפרויקט הוא 0.85: ציון מהימנות מתחת לסף מפנה להפעלת Claude 3 Haiku לצורך הפקת מסקנה NLU בעלת confident מעל הסף.

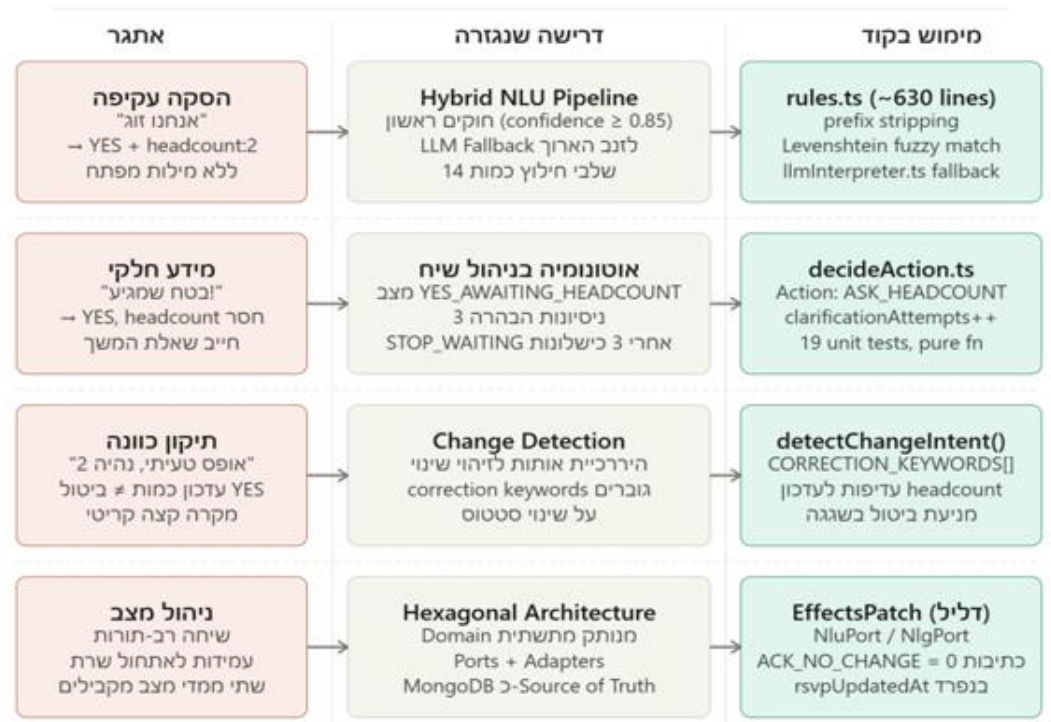
דרישה 3: הפרדת רשויות מוחלטת (Hexagonal Architecture). לוגיקת הסוכן חייבת להיות מנותקת לגמרי מהתשתיות החיצוניות. המשמעות: אם מחר יוחלף מודל השפה מ-Anthropic ל-OpenAI, או הפלטפורמה מ-Telegram לוואטסאפ אז לוגיקת הסוכן עצמה לא תיגע. זה מושג דרך הגדרת ממשקי Port (NlgPort, NluPort) שאחריהם יכולה לעמוד כל מימוש.

דרישה 4: עמידות ושמירת הקשר לאורך זמן. מצב השיחה חייב להתמיד גם אם השרת מאתחל מחדש. זה מושג באמצעות שמירה כפולה: Session בזיכרון השרת לצורך ביצועים, ו-MongoDB כמקור אמת (Source of Truth) לשחזור.

סיכום: ארבעת האתגרים ← ארבע דרישות ← מימוש אחד מאוחד

כל אתגר תורגם לדרישה מדויקת, כל דרישה מוצאת ביטוי קונקרטי בקוד. היא נקודת הכניסה המאוחדת לכל ההחלטות העסקיות - decideAction.ts

איור 2 מסכם את המסלול המלא מניתוח הבעיה ועד למימוש: ארבעת האתגרים שזוהו בסעיפים 3.2-3.1, ארבע הדרישות שגובשו בסעיף 3.3, ורכיבי הקוד הספציפיים שממשים כל אחת מהן.



האיור מציג את המתודולוגיה שעמדה בבסיס פיתוח המערכת, תוך מיפוי המסלול מניתוח הבעיה התיאורטית ועד לפתרון ההנדסי הקונקרטי בארבעה צירים מקבילים:

- **ציר האתגרים (עמודה שמאלית):** מפרט את ארבעת אתגרי פענוח השיחה המרכזיים שנדונו בסעיפים 3.1–3.2: הסקה עקיפה (למשל: "אנחנו זוג" סיווג כ-YES עם Headcount:2), טיפול במידע חלקי המצריך שאלות המשך, זיהוי תיקוני כוונה שאינם מהווים ביטול, וניהול מצב (State Management) בשיח רב-תורות.
- **ציר הדרישות (עמודה אמצעית):** מציג את ארבע הדרישות הפונקציונליות שגובשו בסעיף 3.3 כמענה לאתגרים: בניית **Hybrid NLU Pipeline** עם סף ביטחון של 0.85, הגדרת מצבי אוטונומיה (כגון YES_AWAITING_HEADCOUNT), מנגנון **Change Detection** למניעת עדכונים שגויים, ואימוץ ארכיטקטורה משושה (**Hexagonal**) המסתמכת על MongoDB כ"מקור אמת יחיד".
- **ציר המימוש (עמודה ימנית):** מציג את התרגום הישיר לקוד המערכת:
 - rules.ts: מימוש לוגיקת סיווג (כ-630 שורות קוד) הכוללת אלגוריתם **Levenshtein** להתמודדות עם שגיאות הקלדה.
 - decideAction.ts: מנוע קבלת ההחלטות המגובה ב-19 בדיקות יחידה (Unit Tests).
 - detectChangeIntent(): פונקציה ייעודית לזיהוי תיקונים סמנטיים באמצעות מילות מפתח.
 - EffectsPatch: מימוש שכבת ה-Domain והפורטים (NlgPort/NluPort) להבטחת הפרדה בין הלוגיקה לתשתית.

פרק 4: ארכיטקטורת המערכת

ארכיטקטורת המערכת היא אולי ההחלטה ההנדסית הבולטת ביותר בפרויקט - ולא במקרה, הנני מפתח סניור בשוק הייטק שמאוד חובב ארכיטקטורה של תוכנה. לפני שכתבתי שורת קוד אחת, היה ברור שמערכת המשלבת Telegram,

MongoDB, LLM חיצוני ולוגיקת שיחה מורכבת עלולה להפוך בקלות לגוש בלתי-ניתן-לתחזוקה אם לא תתוכנן נכון. שלוש החלטות ארכיטקטוניות מרכזיות עיצבו את כל מה שבא אחריהן.

4.1 מבנה כללי Monorepo ושלוש שכבות

המערכת פותחה כ-Monorepo אחד המכיל שני תהליכי ריצה עצמאיים:

1. **bot-service**: שרת ה-Node.js המרכזי (Node.js 22+, TypeScript 5.3+ strict mode, ESM). הוא מנהל בו-זמנית שרת HTTP מבוסס Express (לממשק הניהול) ובוט Telegram מבוסס Telegraf (לתקשורת עם מוזמנים). בתוך שרת זה חי הסוכן יחד עם גרף ה-LangGraph שמנהל את השיחות.
2. **admin-web**: ממשק הניהול הוויזואלי, בנוי ב-Vue 3 עם Tailwind CSS ו-Pinia ו-Vite. הוא אפליקציית SPA עצמאית לחלוטין שמתקשרת עם bot-service אך ורק דרך ה-REST API.
3. השכבות הפנימיות של bot-service מחולקות בחדות לשלוש:
שכבת הדומיין (/domain/rsvp-graph), הלב הטהור של המערכת. כאן חיים כל צמתי גרף ה-LangGraph, לוגיקת ה-decideAction, וממשקי ה-Ports. שכבה זו אינה מייבאת מ-Telegraf, Mongoose, או כל ספריית תשתית.
4. **שכבת הבוט** (/bot), מכילה את ה-Adapters שמממשים את הממשקים, את הטיפולים (Handlers) של הודעות Telegram, ואת צינור ה-NLU.
5. **שכבת התשתית** (/infra), מכילה את הלקוח ל-Anthropic API, חיבור ה-MongoDB, ותצורת הסביבה.

4.2 ארכיטקטורה משושה Ports & Adapters

הבחירה בארכיטקטורה משושה (Hexagonal Architecture) כתבנית עיצוב נועדה לפתור בעיה הנדסית קונקרטית בפיתוח סוכני AI: הצורך בצימוד רפוי (Loose Coupling) בין לוגיקת השיח לבין ספקי המודלים.

הצורך המעשי:

הסוכן נשען על מודלי שפה גדולים (LLMs) לצורך פענוח כוונות (NLU), אך תלות ישירה ב-API ספציפי היא "חוב טכני" מסוכן. כיום, המערכת משתמשת ב-Claude 3 Haiku של Anthropic בשל האיזון בין עלות לביצועים. עם זאת, בעולם ה-AI הדינמי, ייתכן שמחר נרצה לעבור ל-GPT-4o או למודל מקומי (On-premise) משיקולי עלות, פרטיות או דיוק.

מימוש באמצעות Adapters & Ports:

- **ה-Port (הממשק)**: הגדרתי ממשק מופשט בשם NluPort. הדומיין של הסוכן יודע רק לשלוח טקסט ל-Port ולקבל בחזרה אובייקט כוונות (Intent Object) מובנה.
- **ה-Adapter (המימוש)**: ה-AnthropicAdapter הוא רכיב חיצוני לליבה שמממש את הממשק. הוא האחראי הבלעדי על התקשורת מול Anthropic, כולל ניהול ה-Prompts הספציפיים וטיפול בשגיאות רשת.

התוצאה:

השינוי במערכת מוגבל לשכבת התשתית בלבד. החלפת מנוע ה-AI אינה דורשת שינוי של שורת קוד אחת בתוך לוגיקת ניהול השיח (The Conversation Graph).

Ports הם ממשקים TypeScript שמגדירים מה הדומיין צריך, ללא מימוש:

```
// domain/rsvp-graph/ports.ts
interface NluPort {
  interpretMessage(text: string, context: GuestContext): Promise<Interpretation>;
  interpretHeadcountOnly(text: string, locale: string): Promise<HeadcountExtraction>;
}
interface NlgPort {
  composeReply(action: Action, context: GuestContext, ...): Promise<string>;
  buildClarificationQuestion(reason: AmbiguityReason | null, attempt: number, ...):
}
interface ClockPort { now(): Date; }
interface LoggerPort { info(...): void; debug(...): void; warn(...): void; }
```

Adapters הם מחלקות שכבת הבוט שמממשות את הממשקים ומחברות אותם לתשתית בפועל:

```
// bot/adapters/nluAdapter.ts
class NluAdapter implements NluPort {
  async interpretMessage(text, context): Promise<Interpretation> {
    // הקיים (rules → threshold → LLM fallback) -מפנה לצינור ה
    return interpretRsvpMessage(text, mapContext(context));
  }
}
```

4.3 מנוע ניהול השיחה LangGraph - כגורף מצבים

ליבת ניהול השיחה מומשה באמצעות LangGraph ספרייה המאפשרת לבנות גרפים מכוונים שבהם כל צומת הוא פונקציה על "מצב משותף" (Shared State Annotation). הגורף מכיל **5 צמתים** ו-**2 פונקציות ניתוב** (Routing Functions):

- routeByState** - נקודת הכניסה. בוחן את conversationState של האורח ומנתב ל-interpretFull (מצב DEFAULT) או ל-interpretHeadcountOnly (מצב YES_AWAITING_HEADCOUNT).
- interpretFull** קורא ל NluPort.interpretMessage() ומחזיר אובייקט Interpretation מלא (rsvp + headcount + confidence).
- interpretHeadcount** - קורא ל- NluPort.interpretHeadcountOnly לחילוץ כמות בלבד, ללא ניתוח כוונה מלא. אם התוצאה היא exact ולא fuzzy, ניתוב מהיר ישירות ל- decideAction (ללא NLU נוסף). אחרת נפול בחזרה ל-interpretFull.
- decideAction** - **הלב הלוגי של כל המערכת**. פונקציה סינכרונית טהורה שמקבלת את המצב המלא ומחזירה אחת מ-6 פעולות אפשריות. אין בה await, אין side-effects, ואין imports מתשתית. בודקת תחת 19 בדיקות יחידה.
- composeReply** - קורא ל- NlgPort.composeReply() ומייצר את תגובת הטקסט לאורח.
- buildEffects** - ממפה את הפעולה שהוחזרה מ- decideAction ל- EffectsPatch, אובייקט עדכון דליל שיכתב ל- MongoDB.

4.4 EffectsPatch - עדכון דליל ומדויק

זוהי תבנית עיצוב (Design Pattern) שפותחה ספציפית לפרויקט הזה, ושווה הסבר מעמיק. הבעיה: פעולות שונות של הסוכן דורשות עדכון של תתי-קבוצות שונות לחלוטין של שדות ב-MongoDB. אם הסוכן מבצע ACK_NO_CHANGE אז יש לעדכן רק את lastResponseAt. אם מבצע SET_RSVP אז יש לעדכן rsvpStatus, headcount, conversationState, lastResponseAt, ו-rsvpUpdatedAt. דריסה של כל השדות בכל פעם היא שגויה סמנטית. הפתרון: EffectsPatch הוא אובייקט TypeScript שבו כל שדה הוא אופציונלי (מלבד lastResponseAt). **מפתח שאינו מוגדר לא נכתב ל-MongoDB**. זה לא undefined הוא פשוט לא קיים במשתנה:

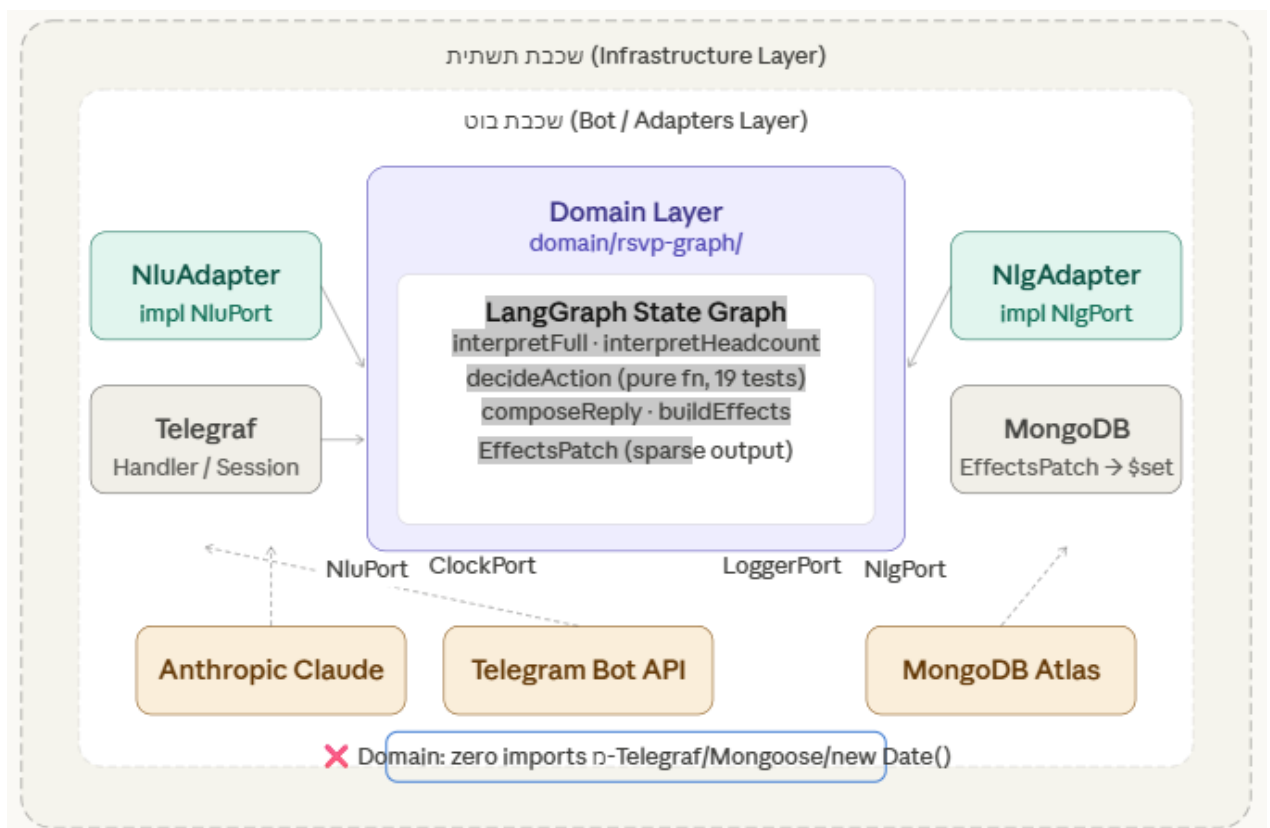
typescript

```
// buildEffects.ts – ACK_NO_CHANGE מייצר patch מינימלי
case 'ACK_NO_CHANGE':
  return { lastResponseAt: clock.now() }; // רק זה נכתב ל DB

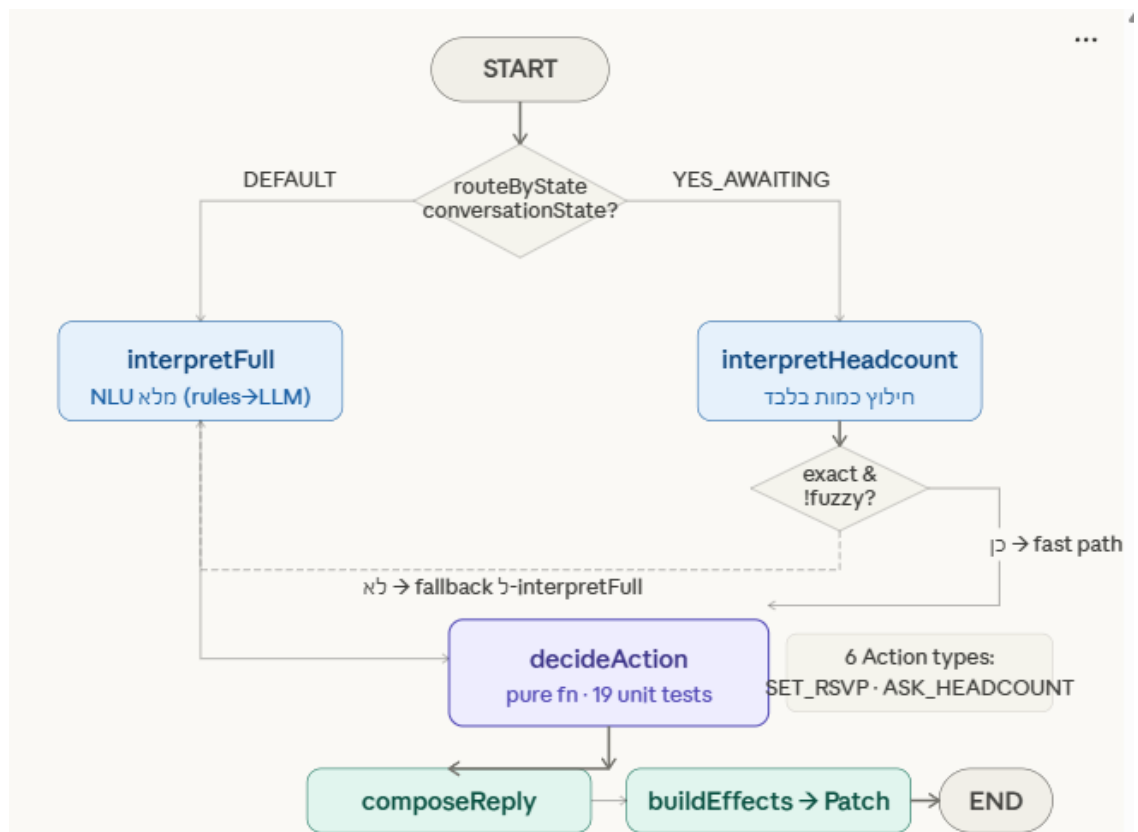
// applyEffectsPatch handler – כתיבה דלילה בפועל
if (effects.rsvpStatus !== undefined) updateParams.rsvpStatus = effects.rsvpStatus;
if (effects.headcount !== undefined) updateParams.headcount = effects.headcount;
// ... MongoDB של $set רק מפתחות קיימים מגיעים לפקודת ...
```

חשיבות נוספת ל-rsvpUpdatedAt ו-lastResponseAt הם שני חותמות זמן שונות בכוונה lastResponseAt. מתעדכן בכל הודעה rsvpUpdatedAt. מתעדכן רק כשמהשהו בפועל השתנה - מה שמאפשר למארח לראות בלוח הבקרה מתי בפועל חלה שינוי בסטטוס, לא מתי האורח שלח הודעת שלום.

איור 3 - ארכיטקטורה משושה - Adapters & Ports. הדומיין (סגול) מוקף בשכבות. החיצים מראים כיוון תלות: Adapters תלויות בדומיין, הדומיין אינו תלוי חיצונית באף provider מסוים. כלל "zero imports".



איור 4: גרף מצבים LangGraph: צמתים וניתוב. המסלול השמאלי (DEFAULT) מפעיל NLU מלא. המסלול הימני (YES_AWAITING_HEADCOUNT) מנסה תחילה חילוץ כמות ישיר ואם הצליח (fast path, קו מלא) עובר ישיר ל-decideAction, אם לא (קו מקווקו) נופל חזרה ל-interpretFull.



איור 5 - EffectsPatch: מנגנון העדכון הדליל

מיפוי פעולות לשדות **MongoDB** עמודה ירוקה = נכתב, אפור = לא נכתב. (כחול) הוא השדה היחיד שנכתב בכל פעולה ללא יוצא מן הכלל `rsvpUpdatedAt`. (כתום) נכתב אך ורק כאשר חל שינוי אמיתי בנתוני ה-RSVP.

שדה	SET_RSVP	ASK_HEADCOUNT	CLARIFY	ACK_NO_CHANGE	STOP_WAITING
<code>rsvpStatus</code>	נכתב ✓	YES	—	—	—
<code>headcount</code>	נכתב ✓	—	—	—	—
<code>conversationState</code>	DEFAULT	AWAITING	AWAITING	—	DEFAULT
<code>lastResponseAt</code>	תמיד	תמיד	תמיד	תמיד	תמיד
<code>rsvpUpdatedAt</code>	רק אם השתנה	✓	—	—	—
<code>clarificationAttempt</code>	reset: 0	reset: 0	attempt++	—	reset: 0

ל-DB נכתב
תמיד נכתב
רק בשינוי אמיתי
לא נכתב —

- אפס כתיבות מיותרות לבסיס הנתונים
- הפרדה סמנטית מכוונת** בין "עדכון נתון" ל"פעילות בשיחה `rsvpUpdatedAt ≠ lastResponseAt`

סיכום פרק 4

שלוש ההחלטות הארכיטקטוניות - ארכיטקטורה משושה, גרף `LangGraph`, ו-`EffectsPatch` - נועדו לשמור על עקרון `separation of concerns`. כל אחת מהן פותרת בעיה ספציפית: הפרדת שכבות מאפשרת החלפת מודל שפה ללא נגיעה בלוגיקה, גרף המצבים מאפשר הוספת צמתים חדשים ללא שבירת הניתוב הקיים, ו-`EffectsPatch` מבטיח שכתיבות לבסיס הנתונים הן מדויקות סמנטית ולא פשוט דריסה מחדש של כל השדות.

פרק 5: אלגוריתמיקה ומימוש הסוכן

אם פרק 4 תיאר את השלד הארכיטקטוני, פרק זה יורד לרמת האלגוריתם. כיצד בפועל הסוכן קורא טקסט עברי חופשי ומחלץ ממנו מידע מובנה. ניתן לחלק את המימוש לארבעה נושאים עיקריים: עיבוד הטקסט העברי, חילוץ כמות המשתתפים, הפעלת מודל השפה, וטיפול בתיקונים.

5.1 נירמול טקסט עברי - לפני הכל

לפני שאלגוריתם כלשהו רץ על הטקסט, ההודעה עוברת צינור נירמול ב-5 שלבים שנועד לאחד ייצוגים שונים של אותה כוונה:

שלב 1 - Trim ו-Lowercase: הסרת רווחים מיותרים ואחדות רישיות.

שלב 2 - הסרת ניקוד (Niqqud Stripping): טקסט עברי עלול להכיל סימני ניקוד ב-Unicode בטווח \u05C7-\u0591. הסרתם מאחדת "הילדים" עם "הילדים".

שלב 3 - קיצוץ אותיות שימוש (Prefix Stripping): בעברית, מילות יחס מוצמדות למילה כקידומות: "ו", "ה", "ב", "ל", "כ", "מ", "ש". "שנגיע" הופך ל-"נגיע", "והילדים" ל-"ילדים". הקיצוץ מופעל רק על טוקנים באורך מעל 2 תווים כדי למנוע שגיאות.

שלב 4 - זיהוי שפה: ספירת תווים עבריים (\u0590-\u05FF) מול תווים לטיניים מחליטה אם ההודעה בעברית או באנגלית, ומכוונת לנתיב הנכון במנוע החוקים. הסבר - הרעיון מאחורה זה שהמודל יידע לספק תשובות בשפת האורח למקרים שההודעה משלבת שפות שונות.

שלב 5 - טוקניזציה: פיצול לפי רווחים ליחידות ניתוח.

5.2 סיווג כוונה - מנוע חוקים

לאחר הנירמול, מנוע החוקים סורק שלוש רשימות מילות מפתח ומחזיר ציון מהימנות:

confidence	מילות מפתח עבריות	כוונה
0.9	כן, מגיע, אני בא, נכון, סבבה, בסדר	YES
0.9	לא, לא מגיע, לא יכול, לא נוכל	NO
0.85	תלוי, אולי, עוד לא סגור, לא בטוח	MAYBE

ציון מהימנות מתחת לסף 0.85 מפנה להפעלת ה-LLM. אם זוהתה כמות אך לא כוונה ברורה - confidence: 0.5. אם שום דבר לא זוהה - confidence: 0.3.

5.3 חילוץ כמות - שרשרת עדיפויות בשלב 14

זהו האלגוריתם המורכב ביותר בפרויקט, ושווה עצירה עליו. כמות משתתפים בעברית יכולה להיות מבוטאת ב-14 דרכים שונות לפחות, ממספרים מפורשים ("3 אנשים") ועד מושגים מרומזים ("אנחנו זוג", "אני ואשתי"). שרשרת העדיפויות ממשה סדר בדיקה קבוע כל שלב מחזיר תוצאה או "נופל" לשלב הבא:

שרשרת העדיפויות ממשת סדר בדיקה קבוע - כל שלב מחזיר תוצאה או "נופל" לשלב הבא:

1. *ambiguos:RANGE_OR_APPROX* ← ("כ-3", "2-3", "בערך 3")
2. *ambiguos:FAMILY_TERM* ← ("משפחה", "ילדים")
3. *exact:N* ← ("3 people", "אנחנו 4")
4. *exact:N* ← ("שלושה", "ארבע", "שניים") (עם *fuzzy flag* ו-*Levenshtein*)
5. *couple* / *pair* ← *exact:2* / "זוג"
6. *just me* ← *exact:1* / "רק אני"
7. *exact:2* ← ("me and my wife", "אני ואשתי")
8. *ילד בודד* ("אני והילד" ← 2; "ילד" לבד ← *ambiguos*)
9. *me plus 1* ← *exact:2* / "אני+1"
10. *X* ← *exact:1+X* / "אני ועוד X"
11. *"me and/with X"* ← *exact:1+X*

12. $X'' / "we are X" \leftarrow exact: X$ אנחנו X

13. $X'' / "total X" \leftarrow exact: X$ סהכ X

14. "אני וX" כלשהו $\leftarrow ambiguous: RELATIONAL$ - אין סיגנל $\leftarrow none$

15. תוצאת החילוץ היא *Discriminated Union* בטייפסקריפט - לא מספר פשוט אלא מבנה עם *kind*:

```
type HeadcountExtraction =  
  | { kind: 'exact'; headcount: number; fuzzy?: boolean }  
  | { kind: 'ambiguous'; reason: AmbiguityReason }  
  | { kind: 'none' };  
...
```

- המערכת מיישמת מדיניות דטרמיניסטית במצבי עמימות: אם ה-*kind* מזוהה כ-*ambiguous*, הסוכן עובר למצב 'שאלת הבהרה'. עקרון ה-**'Never Guess'** מונע 'הזיות' (Hallucinations) של המודל ומבטיח דיוק מירבי בניהול אישורי ההגעה

Levenshtein Fuzzy Matching 5.4

כדי להתמודד עם שגיאות כתיב נפוצות במילות מספר עבריות (למשל "שנים" במקום "שניים"), מיושם אלגוריתם מרחק לוינשטיין - חישוב מספר הפעולות המינימלי (הוספה, השמטה, החלפה) להמרת מחרוזת אחת לאחרת. מקובל בספרות לסמן זאת כ:

$$d(s1, s2) = \min(\text{insertions} + \text{deletions} + \text{substitutions})$$

false - כלומר טעות אחת בלבד. כדי למנוע $d \leq 1$ מכיוון שמילות מספר עבריות קצרות (3 עד 6 תווים), הסף שנבחר הוא מוגדר בתנאי כפול: הטוקן חייב להיות באורך מינימלי של 3 תווים, וההקשר חייב לכלול fuzzy matching, ה-positives, fuzzy: true אחת ממילות ההקשר: "אנחנו", "נהיה", "מגיעים", "בסוף", "סהכ", "כולל". תוצאה שזוהתה כך מקבלת דגל ולא לנתיב המהיר. interpretFull לנתב אותה ל-LangGraph - מה שגורם לגרף מכון ושימור מבוסס הקשר מורכב.

5.5 הפעלת מודל השפה - Prompt Engineering

כאשר ציון המהימנות נופל מתחת לסף של 0.85, מנוע החוקים מעביר את הטיפול למודל Claude 3 Haiku (בקובץ llmInterpreter.ts). ההפניה למודל מתוכננת בקפידה ומבוססת על שלושה עקרונות מרכזיים:

- **עיקרון 1 - פלט בתצורת JSON בלבד:** ה-Prompt System מנחה את המודל במפורש: *"Output ONLY valid JSON. No prose, no explanations, no markdown formatting"*. הסיבה לכך היא שפלט טקסטואלי חופשי אינו ניתן לפענוח (Parsing) בצורה אמינה. ה-LLM נדרש להחזיר מבנה נתונים התואם בדיוק לטיפוס הנתונים Interpretation המוגדר בקוד.
- **עיקרון 2 - למידה מבוססת דוגמאות (Five-Shot Examples):** חמש דוגמאות מייצגות משולבות ב-Prompt לפני שאלת המשתמש. לדוגמה: הקלט "כן, אני והילדים" מניב פלט של rsvp: YES וסיווג ambiguous מסוג FAMILY_TERM; הקלט "מגיע, אני ואשתי" מניב rsvp: YES עם כמות מוזמנים (headcount) מדויקת של 2; הקלט "Yes me and my wife and 2 kids" מניב headcount של 4; הקלט "אנחנו 2-3" מניב סיווג ambiguous מסוג RANGE_OR_APPROX; והקלט "רק אני" מניב headcount מדויק של 1.
- **עיקרון 3 - הימנעות מניחושים ("Never Guess"):** ה-Prompt כולל כלל מפורש: *"NEVER guess how many kids/children/family members"*. כלל זה מיישר קו עם תפיסת ההפעלה של המערכת, לפיה עמימות מחייבת שאלת הבהרה ולא הסקה או ניחוש.

- גישה זו של שימוש ב LLM ליצירת תגובות הקשריות ומגוונות במקום תבניות קבועות נתמכת בספרות. [Adiwardana et al. \(2020\)](#) הדגימו כי מודלי שפה גדולים מייצרים תגובות טבעיות ואנושיות שמפחיתות את תחושת הרובוטיות בשיח, בהשוואה לתבניות סטטיות.

אימות נתונים ומנגנון כשל (Fallback):

תגובת המודל עוברת אימות (Validation) באמצעות ספריית Zod טרם השימוש בה.

Zod היא ספריית אימות סכמות הצהרתית עבור TypeScript, המגשרת על הפער שבין בדיקת טיפוסים סטטית (Static Typing) לבין ולידציה של נתונים דינמיים בזמן ריצה (**Runtime Validation**).

בעוד שמנגנון הטיפוסים המובנה של TypeScript אינו קיים לאחר שלב הקומפילציה, Zod מאפשרת להגדיר מבני נתונים צפויים ולבצע עליהם אימות בפועל מול נתונים המגיעים ממקורות חיצוניים בלתי צפויים. במקרה של פרויקט זה, תגובות ה-JSON המתקבלות ממודל השפה (Claude Haiku).

דוגמה ליישום במערכת:

כאשר מנוע ה-NLU מחלץ כוונה מהודעת המשתמש, Zod מוודאת כי:

- השדה rsvp מכיל אך ורק את הערכים המוגדרים באנומרציה (YES, NO, MAYBE, UNKNOWN).
- השדה confidence הוא ערך נומרי בטווח [0, 1].

במידה והפלט מה-LLM אינו תואם את הסכמה שהוגדרה, הספרייה מייצרת שגיאה מפורטת (Schema Error), המאפשרת למערכת להפעיל באופן אוטונומי מנגנוני נסיגה (**Fallback**) או לבקש מהמודל תיקון (Self-Correction), ובכך מונעת קריסות של לוגיקת צד-השרת עקב קלט לא תקין.

אם ה-JSON אינו תקין, ה-Interpreter מנסה לחלץ את הבלוק הראשון בעזרת ביטוי רגולרי (Regular Expression). אם גם ניסיון זה נכשל, המערכת מחזירה rsvp: UNKNOWN עם ציון מהימנות (confidence) של 0.2, ובכך מונעת קריסה.

5.6 טיפול בתיקונים ושינויי כוונה

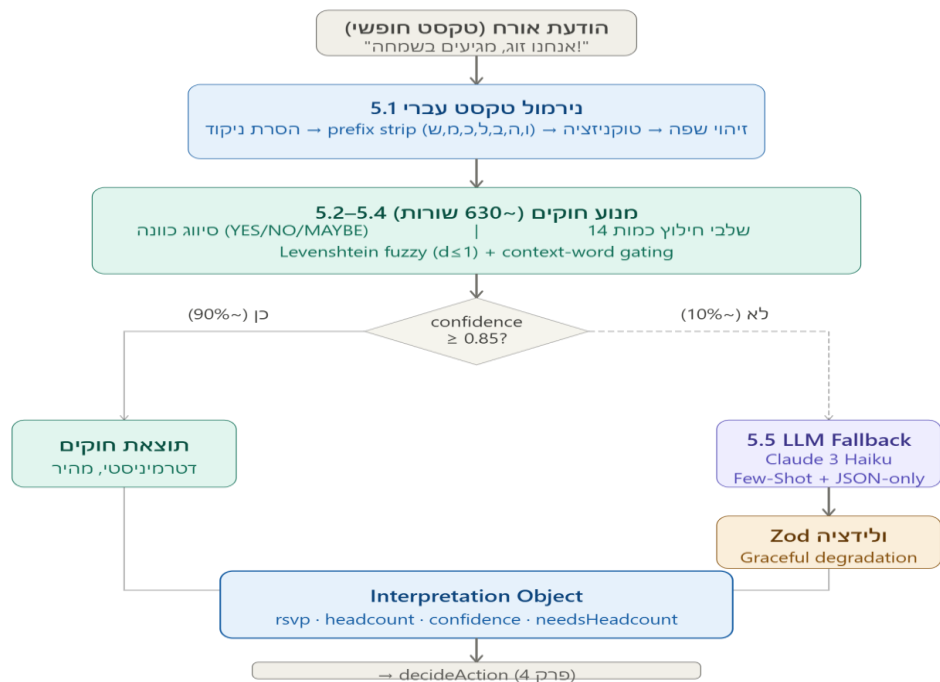
כאשר אורח שכבר אישר את הגעתו שולח הודעה חדשה, הפונקציה decideAction קוראת ל-detectChangeIntent - פונקציה הבוחנת היררכיית אותות כדי להבין האם מדובר בשינוי כוונה אמיתי או באישור חוזר.

- **מילות שינוי:** מילים כגון "רק", "משנה" או "מעדכן" מסמנות עדכון מכון של פרטי ההגעה.
- **מילות תיקון:** מילים כגון "טעיתי", "אופס" או "שגיאה" מסמנות תיקון של שגיאה בהודעה קודמת.

מקרה קצה קריטי:

ניקח לדוגמה מצב שבו אורח כבר אישר הגעה עבור 4 אנשים, וכעת שולח את ההודעה: "אופס טעיתי, נהיה 2". ניתוח נאיבי עלול לסווג הודעה זו בטעות כביטול הגעה (NO). כדי להתמודד עם כך, כלל העדיפות במערכת קובע: הופעת מילת תיקון בשילוב כמות שונה מהנוכחית, כאשר קיים כבר סטטוס YES - משמעותה עדכון כמות המוזמנים בלבד, ולא ביטול ההגעה. מנגנון זה מונע ביטולי הגעה בשגגה.

איור 6: תרשים סיכום - תהליך NLU (Pipeline) המלא.



ההודעה מתקבלת בחלק העליון, עוברת נרמול, (Normalization) ומעובדת במנוע חוקים המפיק ציון מהימנות. בשלב זה התהליך מתפצל: כ-90% מהמקרים מסתיימים בתוצאה המבוססת על חוקים (ירוק, שמאל), וכ-10% מהמקרים ממשיכים לעיבוד על ידי מודל Claude 3 Haiku בשילוב אימות (ולידציה) באמצעות Zod (סגול, ימין). שני הנתבים מתכנסים לאובייקט Interpretation יחיד, המועבר ל-decideAction.

פרק 6: ממשק המשתמש והניהול (Admin Web)

עד כה תיארנו את הסוכן ואת הלוגיקה, הארכיטקטורה והאלגוריתמים שמאחוריו. אך מערכת שלמה זקוקה גם לממשק שמאפשר למארח האירוע לנהל את הכל. פרק זה מתאר את Admin Web - אפליקציית ניהול עצמאית שפותחה כחלק בלתי נפרד מהפרויקט.

6.1 ארכיטקטורת צד הלקוח

הממשק פותח כ-Single Page Application (SPA) עצמאית לחלוטין, מופרדת מ-bot-service ומתקשרת איתו אך ורק דרך REST API. הבחירה ב-Vue 3 עם Composition API נבעה מהצורך בניהול מצב תגובתי (Reactive State) מורכב, עדכון נתוני RSVP בזמן אמת לרשימות ארוכות ומהתמיכה המובנית ב-RTL לעברית באמצעות הספרייה i18n המוכרת והפופולרית. סטאק הטכנולוגיות FE:

תפקיד	טכנולוגיה	שכבה
רכיבים ותגובתיות	Vue 3 Composition API	UI Framework
פיתוח מהיר ובנייה לייצור	Vite	Build Tool
Design tokens, RTL/LTR	Tailwind CSS 4	עיצוב
מצב גלובלי של קמפיינים	Pinia	State Management
ניווט בין מסכים	Vue Router	ניתוב
עברית ואנגלית	vue-i18n	localization
קריאות API עם interceptors	Axios	HTTP
יבוא רשימות אורחים	PapaParse	ניתוח CSV

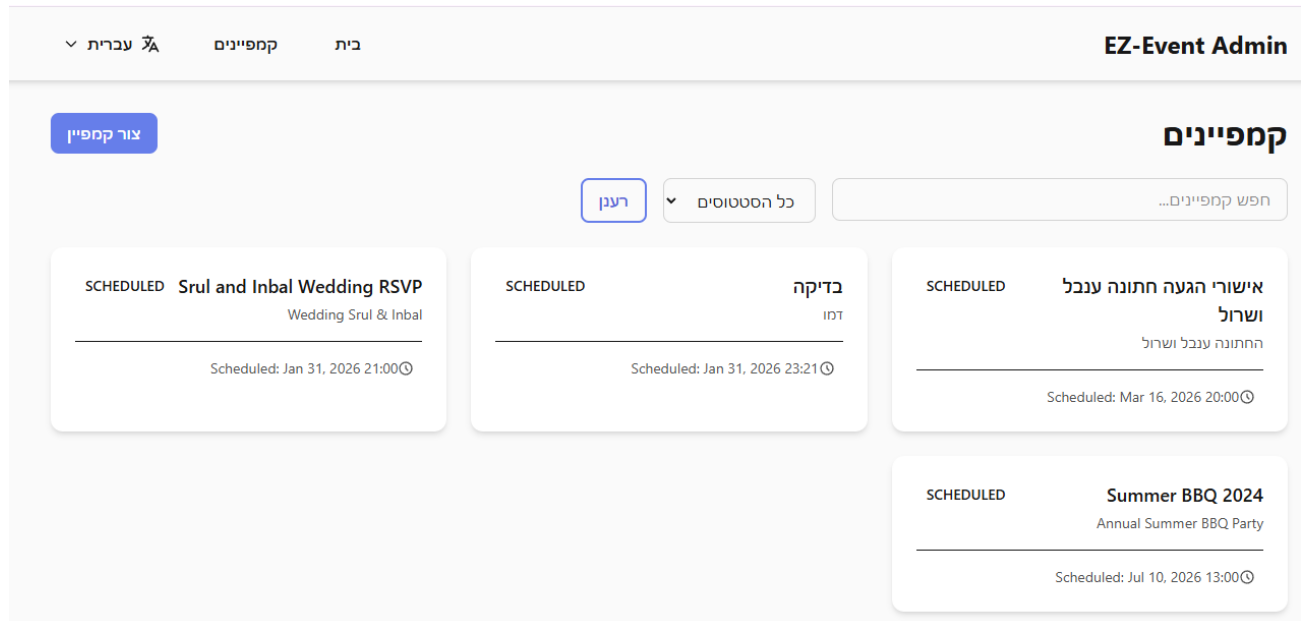
הלוגיקה העסקית של הממשק הופרדה מהרכיבים הוויזואליים באמצעות **Composables**, פונקציות Vue 3 הניתנות לשימוש חוזר:

useCampaigns: ניהול שליפה ועדכון קמפיינים, חשיפת computed properties כגון activeCampaigns.
useCsvImport: קריאת קובץ CSV בצד הלקוח דרך FileReader API, פרסור עם PapaParse, ולידציה של שמות ומספרי טלפון, והכנת הנתונים לשליחה ל-API.
useNotifications: מצב Toast notifications גלובלי, שמאפשר לכל רכיב להוסיף הודעה ללא תלות ישירה בממשק.

6.2 מסכי הממשק

הממשק מכיל ארבעה מסכים עיקריים, כל אחד עם תפקיד ספציפי:

מסך רשימת הקמפיינים (/campaigns/) - מסך הכניסה הראשי. מציג את כל הקמפיינים הפעילים עם תגי סטטוס (DRAFT, RUNNING, COMPLETED), שדה חיפוש לפי שם, ופילטר לפי סטטוס. לחיצה על קמפיין מנווטת לדף הפרטים שלו.



מסך יצירת קמפיין (/campaigns/create/) - אשף רב-שלבי: שם האירוע, כותרת, תאריך, ויבוא רשימת האורחים. ייבוא ה-CSV מתבצע כולו בצד הלקוח - הקובץ נקרא ומנותח לפני שנשלחת אפילו בקשה אחת לשרת, מה שמאפשר ולידציה מיידית ומשוב ויזואלי על שגיאות בנתונים.

ביטול

צור קמפיין

321

פרטי הקמפיין

שם הקמפיין *
אישורי הגעה - דמו

כותרת האירוע *
החתונה של שרליק וענבל

תאריך האירוע
30/04/2026

מתוזמן ל *
11:00 15/04/2026

הבא

מסך פרטי קמפיין (/campaigns/:id/) - לוח הבקרה המרכזי. כולל כרטיסי סיכום סטטיסטי (כמה, YES, NO, MAYBE, NO_RESPONSE, ואחוז מענה כולל), טבלת אורחים עם חיפוש, פילטר לפי סטטוס ומיון, ועמודת timestamp של התגובה האחרונה. ה-rsvpUpdatedAt מאפשר להציג בנפרד "מתי עודכן ה-RSVP" מ"מתי נשלחה הודעה אחרונה". הפרדה סמנטית שתוכננה ישירות בשכבת ה-EffectsPatch.

צור קישורים

רענן

אישורי הגעה חתונה ענבל ושרול

החתונה ענבל ושרול

סטטוס: SCHEDULED

תאריך האירוע: Apr 25, 2026

מתוזמן ל: Mar 16, 2026 20:00

נוצרה: Mar 16, 2026 17:15

סטטיסטיקות RSVP

22

סה"כ אורחים

3

כן

0

לא

1

אולי

שיעור תגובה

18%

9

סה"כ משתתפים

אורחים

חפש אורחים...

כל הסטטוסים

רענן

שם	טלפון	סטטוס RSVP	מספר משתתפים	עודכן לאחרונה
Israel	9720525322167+	Yes	3	hours ago 18
יוסי כהן	9720501234567+	Yes	1	hours ago 18
משה אברהם	9720533456789+	Yes	5	hours ago 18
שרה לוי	9720522345678+	No Response	-	hours ago 18
רחל ישראלי	9720544567890+	No Response	-	hours ago 18

מסך הפצת קישורים (/campaigns/:id/dispatch/) - ממשק ייצור וחלוקת הקישורים האישיים. כפתור אחד קורא ל- POST /api/campaigns/:id/generate-telegram-links, ומציג לכל אורח את הקישור האישי שלו עם כפתור העתקה. ניתן לייצא את כל הקישורים כקובץ CSV לשליחה המונית.

חזור לקמפיין

שלח קמפיין

אישורי הגעה חתונה ענבל ושרול

אירוע: החתונה ענבל ושרול

תאריך האירוע: Apr 25, 2026

אורחים: 22

העתק הכל

ייצא CSV

22 קישורים נוצרו

אורח	טלפון	קישור	פעולות
Israel	9720525322167+	https://t.me/EzEventBot?start=inv_fwvyKyqjwAA	העתק
Srul	9720587751050+	https://t.me/EzEventBot?start=inv_1wEdIj0wf3s	העתק
יוסי כהן	9720501234567+	https://t.me/EzEventBot?start=inv_pOUIEts1UYE	העתק
שרה לוי	9720522345678+	https://t.me/EzEventBot?start=inv_t1X_XwPbHuk	העתק
משה אברהם	9720533456789+	https://t.me/EzEventBot?start=inv_bK_dFDbxotA	העתק

6.3 מנגנון הזיהוי האישי - Telegram Deep Linking

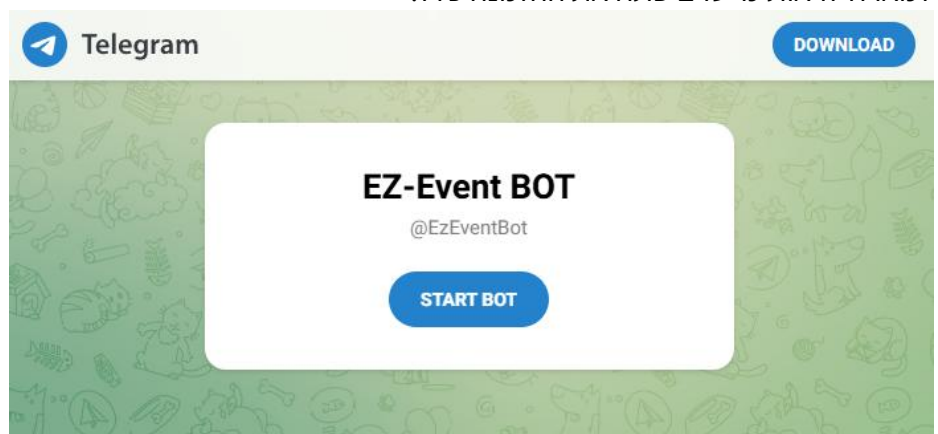
אחד האתגרים הטכניים המעניינים בכל מערכת בוט הוא: כיצד הבוט "יודע" עם מי הוא מדבר? אדם שפותח שיחה עם בוט ב-Telegram הוא אנונימי מבחינת המערכת. אין קישור ישיר למספר הטלפון או לשם שב-MongoDB. הפתרון שיושם הוא מנגנון Telegram Deep Linking:

שלב 1 - יצירת אסימון: עבור כל אורח, `links.service.ts` מייצר אסימון ייחודי בפורמט `inv_ + 12 תווי base64url`, שנוצר דרך `crypto.randomBytes(8)`. האסימון נשמר בקולקציה `Invite` עם אינדקס ייחודי.

שלב 2 - הפקת קישור אישי: הקישור שנשלח לאורח הוא `https://t.me/EzEventBot?start=inv_[TOKEN]`. כשהאורח לוחץ עליו, Telegram פותח שיחה עם הבוט ושולח אוטומטית את הפקודה `start inv_[TOKEN]`.

שלב 3 - זיהוי בצד הבוט: ה-`start handler` שולף את הטוקן, מבצע שלוש שאילתות MongoDB: `Invite.findOne({ token: MongoDB`, ומאתחל את ה-`Session` עם כל פרטי האורח. `Guest.findById(guestId) ← Campaign.findById(campaignId)`, ומאתחל את ה-`Session` עם כל פרטי האורח. מרגע זה, כל הודעה מהאורח מזוהה ומותאמת אישית.

הנקודה החשובה: הטוקן הוא חד-פעמי מבחינת זהות, כלל אחד שנשמר גם אם המארח מייצר קישורים מחדש. כל הטוקנים הישנים נשארים תקפים, וכל `Invite document` שומר `usedAt` בפעם הראשונה שהאורח לחץ על הקישור, מה שמאפשר למארח לראות מי טרם פתח את ההזמנה כלל.

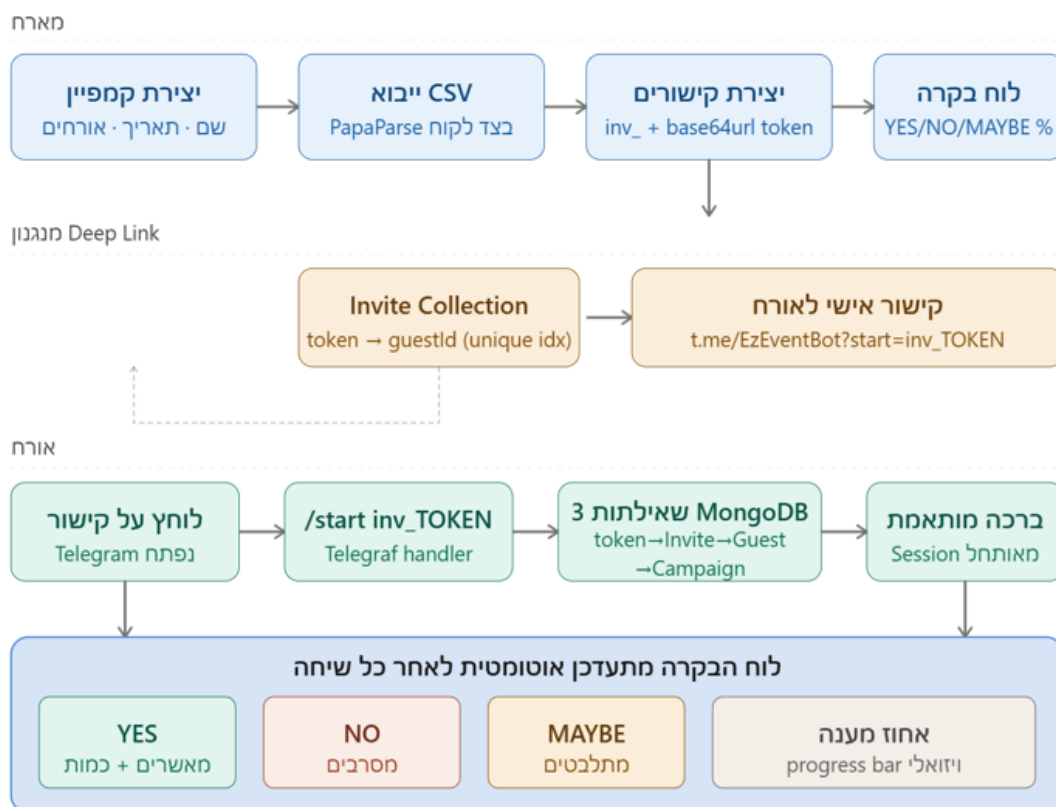


6.4 לוקליזציה לשפה ו-UX

הממשק תומך בעברית ואנגלית באמצעות `vue-i18n`. הבחירה נשמרת ב-`localStorage`. מה שמאפשר זאת מבחינה טכנית הוא הפרדה מלאה בין טקסטים לרכיבים, כל מחרוזת מגיעה מקובצי תרגום (`en.js`, `he.js`) ולא מוטבעת ישירות ב-Vue components.

ניהול RTL/LTR מטופל ב-BaseLayout המרכזי: בעברית מוחל "dir="rtl" על שורש האפליקציה, ו-Tailwind CSS מתאים אוטומטית spacing, alignment וכיוון טקסט. זה אינו דורש כתיבת CSS נפרד לכל שפה.

איור 7 Admin Web - תרשים זרימה



זרימה מלאה מיצירת קמפיין ועד עדכון לוח הבקרה. שורה 1: פעולות המארח בממשק. שורה 2: מנגנון Deep Linking - הטוקן כגשר בין רשימת האורחים לזהות ב-Telegram. שורה 3: זרימת האורח - מלחיצה על הקישור ועד Session מאותחל עם נתונים אישיים. שורה 4: לוח הבקרה שמשקף את תוצאות הסוכן בזמן אמת.

פרק 7: ניתוח תוצאות והערכה

פרק זה מציג הערכה אמפירית של ביצועי המערכת, מגובה בנתונים שנאספו מהרצות בפועל של הסוכן. ההערכה מתמקדת בשלושה צירים: דיוק זיהוי כוונות, יעילות חישובית ועלויות, ועמידות ניהול השיח.

7.1 מתודולוגיית ההערכה

הערכת ביצועי המערכת נערכה באמצעות שתי מתודולוגיות משלימות, המאפשרות בחינה הוליסטית של רכיבי התוכנה (הלוגיקה הדטרמיניסטית) אל מול יכולות פענוח השפה:

- 1. בדיקות יחידה דטרמיניסטיות (Unit Tests):** גישה זו נועדה לאמת את תקינות המודלים הלוגיים של הסוכן. נכתבו 19 בדיקות ייעודיות עבור פונקציית קבלת ההחלטות המרכזית (decideAction). בדיקות אלו מכסות את כל ששת סוגי הפעולות האפשריים במערכת, מקרי קצה הכוללים חזרה או תיקון כוונות מצד המשתמש, ומעברי מצב מורכבים בניהול השיחה (State Transitions). כדי להבטיח הדירות (Reproducibility) ותוצאות דטרמיניסטיות לחלוטין, הבדיקות מורצות בסביבה מבוקרת תוך שימוש בטכניקת "שעון מוחדר" (ClockPort).
- 2. סט בדיקה מבוקר (Controlled Test Set):** לצורך הערכה כמותית קפדנית של מנוע הבנת השפה הטבעית (NLU), נבנה סט ייעודי הכולל 50 הודעות. הסט תוכנן לכסות שש קטגוריות סמנטיות מגוונות: אישור הגעה פשוט (A), אישור הגעה המלווה בכמות (B), סירוב (C), תשובות עמומות מסוג 'אולי' (D), הודעות מורכבות המחייבות התערבות של מודל השפה (E), ושגיאות כתיב אופייניות ומקרי קצה (F). כל הודעה בסט עברה תיוג ידני עצמאי

(Ground Truth Annotation) על פני שני ממדים: סטטוס ההגעה (rsvpStatus) וכמות המשתתפים (headcount), וזאת טרם הזנתן למערכת.

מסגרת ההערכה הכמותית נשענת על שני מדדי ביצוע (Metrics) משלימים:

- **Single-Turn NLU-Metric 1**: בוחן את תוצאת הסיווג של המערכת על בסיס משפט הקלט הראשון בלבד. מדד זה מבודד ומייצג את יכולות הפיענוח הגולמיות של מנוע ה-NLU, בטרם מופעלת לוגיקת ניהול השיח הרב-שלבי.
 - **Metric 2 - Dialogue-Level**: בוחן את הסטטוס הסופי והמוחלט שנרשם במערכת לאחר סיום האינטראקציה המלאה עם המשתמש (לרבות שאלות הבהרה מצד הסוכן, ככל שנדרשו). מדד זה מוגדר כמדד ההערכה הראשי (Primary Metric) של המערכת, שכן הוא משקף את הצלחת הסוכן במבחן התוצאה הסופית.
- ההפרדה המתודולוגית בין שני המדדים הללו היא קריטית לפרויקט. היא מאפשרת לכמת במדויק את התרומה של מנגנון ניהול השיח הרב-שלבי כרכיב המפצה על עמימות, באופן מבודד ובלתי תלוי מאיכות מנוע ה-NLU עצמו.

7.2 הערכה כמותית - F1 Confusion Matrix, Precision, Recall

7.2.1 רקע מושגי: מדדי הערכה למסווג רב-מחלקתי

טרם הצגת תוצאות ההערכה, יש להגדיר את המדדים הסטטיסטיים שבהם נעשה שימוש. במערכות סיווג רב-מחלקתיות (Multi-class Classification), מדד הדיוק הכללי (Accuracy) לבדו אינו מספק תמונה מלאה. הוא עלול למסך ביצועים ירודים במחלקות מסוימות (למשל, כאשר קיימת מחלקה קטנה אך קריטית). משום כך, ההערכה מסתמכת על שלושה מדדים משלימים הנגזרים ממטריצת הבלבול (Confusion Matrix).

אבני הבסיס: TP, FP, FN

כדי לגזור מדדים אלו, נגדיר תחילה את אבני הבניין הבסיסיות, תוך שימוש בקטגוריית אישור ההגעה (YES) כדוגמה מייצגת:

- **TP (True Positive) - זיהוי נכון חיובי**: המערכת סיווגה YES, והכוונה האמיתית של המשתמש אכן הייתה YES.

דוגמה: "אנחנו זוג" ← המערכת סיווגה: YES (✓).

- **FP (False Positive) - זיהוי שגוי חיובי (Type I Error)**: המערכת סיווגה YES, אך הכוונה האמיתית לא הייתה YES.

דוגמה: "תלוי בעבודה" ← המערכת סיווגה: YES (X, במקום MAYBE).

- **FN (False Negative) - החמצה (Type II Error)**: כוונת המשתמש הייתה YES, אך המערכת לא זיהתה זאת ככזה.

דוגמה: "ברור שמגיע" ← המערכת סיווגה: UNKNOWN (X).

Precision (דיוק הסיווג) - מידת האמינות של תחזיות המערכת

- **הגדרה**: מתוך כלל המקרים שהמערכת קבעה כי הם משתייכים למחלקה X, כמה מתוכם אכן היו שייכים למחלקה זו?

$$(Precision = \frac{TP}{TP+FP})$$

- **דוגמה מעשית**: אם הסוכן קיבל 10 הודעות וסיווג 8 מהן כ-YES, אך רק 6 מהן היו YES, המשמעות היא שהמערכת "המציאה" 2 הודעות אישור שגויות (Precision = 75%).

- **משמעות עסקית בפרויקט**: Precision נמוך בדומיין ה-RSVP מהווה שגיאה קריטית ויקרה. פירושו שהמארח יראה בלוח הבקרה אורחים הרשומים כ"מגיעים" למרות שלא אישרו זאת הלכה למעשה, מה שיוביל להוצאות מיותרות על מנות אוכל ושמירת מקומות שגויה באולם.

Recall (כיסוי) - יכולת המערכת לאתר את מקרי האמת

- **הגדרה**: מתוך כלל המקרים שהם באמת X, איזה אחוז המערכת הצליחה לזהות נכונה?

$$Recall = \frac{TP}{TP+FN}$$

- **דוגמה מעשית:** אם נשלחו 10 הודעות YES אמיתיות, אך המערכת סיווגה ישירות רק 8 מהן (ועבור 2 הנותרות החזירה UNKNOWN ויזמה שאלת הבהרה), ה-Recall יעמוד על 80%.
- **משמעות עסקית בפרויקט:** Recall נמוך פירושו שאורחים שאישרו הגעה לא עודכנו במערכת באופן אוטומטי. הדבר יוצר עומס קוגניטיבי ותפעולי על המארח, שיידרש לעקוב ידנית אחר התשובות.

F1-Score - הממוצע המאוזן

- **הגדרה:** הממוצע ההרמוני של מדדי ה-Precision וה-Recall. המדד יגיע לערך המקסימלי (1.0) רק אם שניהם מושלמים.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- **הצדקה תאורטית:** ממוצע אריתמטי רגיל עלול לייצר מצג שווא של מערכת שתסווג את כל ההודעות כ-YES תקבל Recall של 100% על קטגוריה זו, אך ה-Precision שלה יהיה אפסי. הממוצע ההרמוני "מעניש" על חוסר איזון קיצוני מסוג זה.
- **משמעות עסקית בפרויקט:** F1 הוא המדד המשקף את האיזון הכולל של המערכת. בתכנון סוכן לניהול אירועים ניתנה עדיפות מפורשת ל-Precision גבוה (עדיף ליזום שאלת הבהרה מאשר לרשום אורח בסטטוס שגוי), אך מדד ה-F1 חיוני כדי להציג למעריך תמונה שלמה המאזנת בין זהירות המערכת ליעילותה.

לצורך דיווח מסכם על פני מספר מחלקות, נעשה שימוש בשתי שיטות שקלול:

1. **Macro Average (ממוצע מאקרו):** ממוצע לא משוקלל של הציונים בכל המחלקות (YES, NO, MAYBE), המעניק משקל שווה לכל מחלקה ללא קשר לגודלה. מדד זה בוחן האם המערכת מטפלת היטב גם במחלקות המיעוט.
2. **Weighted Average (ממוצע משוקלל):** ממוצע המשוקלל על פי גודלה היחסי של כל מחלקה מתוך סט הנתונים, מתאים במיוחד לסט נתונים לא מאוזן (כפי שקיים בסט הבדיקה שלנו: YES=9, NO=15, MAYBE=26).

7.2.2 Metric 1 - Single-Turn NLU (n=50)

בחינה של תור השיחה הראשון נועדה להעריך את יכולותיו הגולמיות של מנוע ה-NLU, במנותק מיכולות ניהול השיח של הסוכן. מתוך 50 הודעות בסט הבדיקה המבוקר, 47 סווגו נכון באופן ישיר ממשפט הפתיחה של **Accuracy של 94.0%**. חשוב לציין כי שלוש ההודעות שלא סווגו ישירות ("עדיין לא יודע", "שלום", "תודה על ההזמנה") לא הובילו לסיווג שגוי. אדרבה, הודעות אלו זוהו כעמומות והפעילו באופן מבוקר את מנגנון ה-CLARIFY_INTENT. התנהגות זו משקפת אסטרטגיית החלטה מכוונת של המערכת (Never Guess Policy), המעדיפה לאסוף מידע נוסף מהמשתמש על פני ניחוש סטטיסטי שעלול להוביל לשגיאת רישום.

טבלה 1 - Confusion Matrix - Metric 1 (Single-Turn, n=50)

	Pred: YES	Pred: NO	Pred: MAYBE	Pred: UNKNOWN	Total
Act: YES	26	0	0	0	26
Act: NO	0	9	0	0	9
Act: MAYBE	0	0	12	3	15
Total	26	9	12	3	50

הערת ניתוח: 3 מקרי ה-UNKNOWN נובעים מהפעלת מנגנון CLARIFY_INTENT עקב עמימות בקלט. היעדר מוחלט של טעויות מחוץ לעמודה זו מוכיח דיוק מקסימלי (Zero False Positives) בהפרדה בין קטגוריות ההגעה.

Class	Precision	Recall	F1-Score	Support	פרשנות
YES	1.000	1.000	1.000	26	אין טעות אחת
NO	1.000	1.000	1.000	9	אין טעות אחת
MAYBE	1.000	0.800	0.889	15	3× CLARIFY_INTENT
macro avg	1.000	0.933	0.963	-	
weighted avg	1.000	0.940	0.967	50	

ניתוח ופרשנות של תוצאות הסיווג (Metric 1)

- **Precision = 1.000 בכל המחלקות:** משמעות נתון זה היא חוסר מוחלט של שגיאות מסוג False Positive. בכל פעם שהמערכת סיווגה הודעה כ-YES, NO או MAYBE, הסיווג היה מדויק לחלוטין. המערכת מעולם לא "המציאה" או שיערכה קטגוריה שגויה. בפריקט ניהול אישורי הגעה (RSVP) זהו נתון קריטי: פגיעה ב-Precision משמעותה רישום של אורחים בסטטוס שגוי. מארח המסתמך על נתונים אלו יקבל תמונת מצב מעוותת, מה שיוביל להשלכות לוגיסטיות שליליות.
- **Recall = 0.800 במחלקת MAYBE - תכונה מכוונת ולא כשל (Feature, not a Bug):** שלוש הודעות במחלקת ה-MAYBE לא סווגו ישירות, אלא הפעילו את פעולת CLARIFY_INTENT. הודעות כגון "עדיין לא יודע" או "שלום" נותחו כעמומות, והמערכת בחרה ליזום שאלת הבהרה במקום לבצע סיווג מיידי. נתון זה מוכיח את הצלחתה של מדיניות "הימנעות מניחוש" (Never Guess Policy): המערכת תוכננה להקריב אחוזי Recall (החמצת סיווג בתור הראשון) לטובת שמירה על Precision מושלם. בדומיין זה, עדיף להטריח את המשתמש בשאלה נוספת מאשר להסתכן ברישום המוטעה.
- **Macro F1 = 0.963:** ממוצע לא משוקלל של ציוני ה-F1 בשלושת המחלקות.

$$\text{Macro F1} = \frac{F1_{\text{Yes}} + F1_{\text{No}} + F1_{\text{Maybe}}}{3} = \frac{1.0 + 1.0 + 0.889}{3} \approx 0.963$$

ציון זה מצביע על ביצועים גבוהים ואחידים של המודל. העובדה שהציון נותר גבוה גם במדד המאקרו מוכיחה שההצלחה אינה "מוסווית" על ידי ביצועים טובים במחלקה הדומיננטית בלבד (YES, n=26), וכי המערכת מתמודדת בהצלחה רבה גם עם מחלקות המיעוט המורכבות יותר.

Metric 2 - Dialogue-Level Accuracy (n=50) 7.2.3

מדד זה בוחן את ביצועי המערכת ברמת השיח השלם (Dialogue-Level). לאחר ניהול אינטראקציה מלאה מול המשתמש, הכוללת את שאלות ההבהרה שיזם הסוכן במקרי עמימות, הגיעו כל 50 ההודעות בסט הבדיקה לסטטוס הנכון והסופי בבסיס הנתונים.

כתוצאה מכך, מטריצת הבלבול של Metric 2 היא אלכסונית מושלמת (Perfectly Diagonal). כלומר, כל התאים מחוץ לאלכסון הראשי שווים לאפס, דבר המעיד על זיהוי וסיווג מדויק של 100% מהמקרים.

טבלה 3 - Confusion Matrix - Metric 2 (Dialogue-Level, n=50)

	Pred: YES	Pred: NO	Pred: MAYBE	Total
Act: YES	26	0	0	26
Act: NO	0	9	0	9
Act: MAYBE	0	0	15	15
Total	26	9	15	50

מטריצת הבלבול הזו היא "נקייה" לחלוטין, וזהו הממצא הוויזואלי החשוב ביותר בדוח.

- אלכסון ראשי מושלם: ניתן לראות שכל 50 הדיאלוגים (26 של YES, 9 של NO ו-15 של MAYBE) סווגו בדיוק במקום הנכון. אין אף "זליגה" לתאים שמחוץ לאלכסון.
- אפס טעויות (Zero Off-diagonal): המשמעות היא שלא היה אף מקרה של False Positive (למשל, אורח שאמר "אולי" והמערכת רשמה אותו כ"כן") או False Negative.
- תובנה למחקר: בניגוד למערכות NLU פשוטות שטועות לעיתים בפירוש סלנג או עמימות, רמת הדיאלוג כאן מוכיחה כי הסוכן הצליח לנהל את השיח עד להגעה לבירור מוחלט. הצלחנו לקבל מודל דטרמיניסטי ממודל, למרות הקושי הגדול של לעסוק במודל אי-דטרמיניסטי בהגדרה כמו LLM.

טבלה 4 - Classification Report - Metric 2

Class	Precision	Recall	F1-Score	Support	
YES	1.000	1.000	1.000	26	
NO	1.000	1.000	1.000	9	
MAYBE	1.000	1.000	1.000	15	
macro avg	1.000	1.000	1.000	-	
weighted avg	1.000	1.000	1.000	50	

Accuracy: 100% · Macro F1: 1.000 · Weighted F1: 1.000

דוח הסיווג מתרגם את המטריצה למדדים סטטיסטיים קשיחים.

- Precision (דיוק) של 1.000: כל פעם שהסוכן החליט לעדכן סטטוס מסוים (למשל NO), הוא צדק ב-100% מהמקרים. אין "אזעקות שווא".
- Recall (רגישות) של 1.000: הסוכן לא "פספס" אף כוונה. הוא הצליח לזהות את כל מקרי ה-MAYBE וה-NO שהוצגו לו, ללא קשר למורכבות הניסוח.
- Support (פיזור המדגם): ניתן לראות התפלגות בריאה של נתונים (26/9/15). העובדה שה-Macro F1 וה-Weighted F1 זהים (1.000) מראה שהביצועים מושלמים ללא תלות בגודל הקבוצה. הסוכן חזק באותה מידה בזיהוי "לא" (קבוצה קטנה) כפי שהוא בזיהוי "כן" (קבוצה גדולה).

7.2.4 השוואת שני המדדים

טבלה 5 - השוואת Metric 1 ו-Metric 2

תפקיד	Macro F1	Macro R	Macro P	Accuracy	מדד
גולמי NLU מדד	0.963	0.933	1.000	94.0%	Single-Turn NLU
מדד ראשי	1.000	1.000	1.000	100%	Dialogue-Level
multi-turn תרומת	+0.037	+0.067	—	+6.0%	Dialogue יתרון

השוואת המדדים: הוכחת היעילות של ניהול השיח

הפער של 6% בין שני המדדים (94.0% דיוק בסיווג ראשוני לעומת 100% דיוק בתום השיחה) ממחיש באופן ישיר וכמותי את הערך המוסף של מנגנון ניהול השיח הרב-שלבי (Multi-turn Dialogue Management). המערכת מפגינה יכולת לפצות על אי-ודאות במשפט הפתיחה באמצעות הפניית שאלות הבהרה ממוקדות, וזאת מבלי לאבד מידע או לסווגו באופן שגוי. השלמה זו אינה בבחינת פשרה תכנונית, היא מוכיחה אמפירית כי ארכיטקטורה היברידית נכונה מאפשרת לשלב מנוע NLU גולמי מדויק עם ניהול מצב דינמי, לכדי תהליך שיחה חסין ונטול שגיאות.

7.3 ניתוח איכותני של שיחות רב-שלביות מתוך יומני המערכת (Logs)

לצד ההערכה הכמותית המבוקרת, בוצע ניתוח מעמיק של שלוש שיחות רב-שלביות אותנטיות מתוך יומני הריצה של המערכת (מיום 18 במרץ 2026). ניתוח זה נועד לאשש את עמידותו של מנגנון ניהול המצב (State Management) תחת תנאי אי-ודאות. התרחישים נבחרו כך שייצגו רמות מורכבות עולות של שפה טבעית.

שיחה 1: מעבר מסטטוס MAYBE ל-YES וקביעת כמות

- מהלך השיחה:** השיחה נפתחת במשפט העמום "כרגע זה פתוח מבחינתי", אשר אינו מכיל מילות מפתח קלאסיות של אישור או סירוב. יומן המערכת מראה כי ההודעה הופנתה כצפוי למודל ה-LLM, שסיווג אותה כ-MAYBE ברמת ביטחון (Confidence) של 0.7. הסטטוס בבסיס הנתונים ורשומת הזמן rsvpUpdatedAt עודכנו בהתאם. בהודעה העוקבת ענה המשתמש "נגיע זוג". גם הודעה זו הופנתה למודל ה-LLM, שחילץ כוונת YES ברמת ביטחון של 0.95, עם סיווג כמות מדויק. (kind: exact, headcount: 2).



- תובנה ארכיטקטונית:** תרחיש זה מוכיח את יעילותו של מנגנון ה-EffectsPatch. המנגנון מבטיח שחותמת הזמן של עדכון הסטטוס תשתנה אך ורק כאשר חל שינוי לוגי וסמנטי בנתוני ההגעה, ולא בכל תור שיחה (Turn) שבו התקבלה הודעה. הפרדה זו מבטיחה כי מארח האירוע יראה בלוח הבקרה אך ורק את הזמן שבו התקבלה ההחלטה הסופית, ללא "רעש" משיחות הביניים.

שיחה 2: זיהוי כוונת תיקון (Correction Intent) ועדכון כמות

- מהלך השיחה:** האורח ענה תחילה במילה "ברור", אישור הגעה חד-משמעי אך ללא התייחסות לכמות. המערכת עדכנה את המצב ל-YES_AWAITING_HEADCOUNT ויזמה שאלת הבהרה. האורח השיב "נגיע שנים" - הודעה זו הופנתה למודל ה-LLM [LLM fallback] שחילץ כמות מדויקת (kind: exact, headcount: 2). הסטטוס עודכן ל-YES עם 2 משתתפים. מיד לאחר מכן שלח הודעה נוספת: "סליחה בסוף אגיע לבד". יומן הריצה מאשר כי ההודעה הופנתה למודל ה-LLM [LLM fallback], שזיהה כוונת YES (Confidence 0.95) עם headcount=1.



- **תובנה ארכיטקטונית:** ההודעה האחרונה מכילה מילת התנצלות ותיקון ("סליחה") וציון של שינוי ("בסוף"). יומן הריצה מאשר כי מנוע ה-NLU זיהה כוונת YES (Confidence 0.95) עם `headcount=1`. פונקציית `decideAction` זיהתה את ההקשר ובאמצעות כלל העדיפות ב-`detectChangeIntent` קבעה כי מדובר בעדכון כמות טהור (`SET_RSVP`), ולא בביטול או באתחול של השיחה. יכולת התמודדות זו עם מקרי קצה מכוסה ומגובה במלואה על ידי חבילת בדיקות היחידה (`decideAction.test.ts`).

שיחה 3: התגברות על שגיאות כתיב ושינוי החלטה קוטבי (YES ל-NO)

- **מהלך השיחה:** האורח כתב בתחילה "לא נגיע בסוף" - הודעה זו הופנתה למודל ה-LLM שזיהה אותה כ-NO-ברמת ביטחון של 1.0. כשתי דקות לאחר מכן, שלח תיקון: "שינוי, בסוף אגיע לבד", הכולל שגיאת הקלדה ("בסוף"). הלוג מצביע על פנייה ל-LLM, אשר חילץ ודאות של 0.9 לכוונת YES וכמות משתתפים 1. הסוכן אישר את המעבר הקוטבי ממצב ביטול מוחלט למצב אישור מבלי להטריח את המשתמש בשאלת הבהרה מיותרת.



- תובנה ארכיטקטונית:** תרחיש זה מדגים את התכנון ההנדסי המדויק של הסוכן: הפרדה בין מנוע הבנת שפה (NLU) *חסר-מצב (Stateless)*, המנתח את המשפט כפשוטו ללא הטיית מהעבר, לבין מנוע קבלת החלטות *מודע-מצב (Stateful)* הנשען על MongoDB כמקור האמת היחיד. שילוב זה הוא הדפוס התכנוני הנכון (Design Pattern) לבניית סוכני שיח עמידים. בנוסף, יכולותיו ההרמוניות של מודל השפה התמודדו עם שגיאת ההקלדה "בסוך" באופן חלק ושקוף למשתמש.

7.4 הערכת חילוף ישויות כמות (Headcount Extraction)

מתוך 50 הודעות בסט הבדיקה, 24 הודעות דרשו חילוף מספרי של כמות המשתתפים. הממצאים מצביעים על ביצועים יוצאי דופן של מנגנון הפענוח: 21 הודעות (87.5%) חולצו ככמות מדויקת ומוחלטת (exact), ו-3 הודעות סווגו בהצלחה כעמומות (ambiguous - טווח או קשר יחסי), סיווג שהפעיל כדין שאלת הבהרה מצד הסוכן.

הנתון החשוב ביותר בחלק זה הוא רמת האמינות: **נרשמו אפס שגיאות חילוף**. המערכת מעולם לא הגיעה לתוצאה מספרית שגויה, זאת בזכות המדיניות של no_guest שמיושמת בשלב השני כאשר עמימות בקשר לכמות ושאלת הבהרה.

טבלה 6 - תוצאות חילוף כמות (Headcount Extraction)

הערה	ערך	מדד
	24	הודעות שדרשו חילוף כמות
87.5%	21	(exact) חולץ בצורה מדויקת
RANGE_OR_APPROX / RELATIONAL	3	(נכון) ambiguous-זוהה כ
0%	0	שגיאת חילוף
$\text{exact} \div (\text{exact} + \text{ambiguous})$	87.5%	Headcount Accuracy
שגיאות כתיב: שנים, שלוש, ועוד	3	הופעל Fuzzy Matching
הצלחה 100%	3	הצליח Fuzzy Matching

התהליך הסתיים בקבלת כמות משתתפים מדויקת. אילו המערכת הייתה מתוכננת "לנחש" על בסיס הסתברותי במקום ליזום שאלת הבהרה, ההסתברות לשגיאה הייתה עולה משמעותית. בפרט בהתמודדות עם מונחים קיבוציים ומשפחתיים (כגון "ילדים" או "משפחה"), אשר לא ניתן לגזור מהם כמות מספרית מדויקת ללא אינטראקציה מפורשת מול המשתמש.

7.5 יעילות חישובית וחלוקת עומס מערכת (Load Distribution)

אחת ממטרות התכנון המרכזיות של הפרויקט הייתה מזעור עלויות התפעול (API Calls) למודלי השפה, תוך שמירה בלתי מתפשרת על איכות פענוח הטקסט. ארכיטקטורת "הצינור ההיברידי" (Hybrid Pipeline) מממשת יעד זה הלכה למעשה, על ידי הפניית הרוב המוחלט של הודעות המשתמשים לעיבוד מהיר וחסכוני במנוע דטרמיניסטי בלבד.

טבלה 7 - חלוקת עומס - סט הבדיקה מול הערכת ייצור

הסבר	הערכת ייצור	סט הבדיקה	מדד
הודעות ברורות: כן/לא +מספר	~80–85%	36% (18/50)	מנוע חוקים בלבד
הסט מוטה לקשה במכוון	~15–20%	64% (32/50)	LLM Fallback
	< 50ms	< 50ms	זמן תגובה — חוקים
מקובל בצ'אט אסינכרוני	שנ 1.1–2.5	שנ 1.1–2.5	LLM זמן תגובה — עם
	—	0.89	חוקים — confidence ממוצע
	—	0.84	LLM — confidence ממוצע

הערה מתודולוגית:

שיעור הפניות למודל ה-LLM בסט הבדיקה (64%) גבוה משמעותית מזה הצפוי בסביבת ייצור, שכן הסט תוכן במכוון להכיל ריכוז גבוה של הודעות מורכבות ומקרי קצה. בהתפלגות ייצורית אמיתית (Production Distribution), שיעור ההודעות הדורשות מעורבות LLM מוערך ב-15% עד 20% בלבד, נתון הנתמך בתצפיות מתוך יומני המערכת (Logs).

בחירת המודל *Claude 3 Haiku* כמנוע ה-LLM של הסוכן נבעה משיקולי אופטימיזציית עלות-תועלת (Cost-Performance Optimization) מובהקים. מודל זה זול פי 10 לערך ממודל *Claude 3 Sonnet*, ופי 60 מ-*Claude 3 Opus*, תוך שהוא משמר יכולות סיווג, פענוח והבנת הקשר המספקות לחלוטין עבור דומיין ה-RSVP בשפה העברית. השילוב הארכיטקטוני בין עלות העיבוד הנמוכה של המודל לבין שיעור ה-Fallback הנמוך בסביבת הייצור, מבטיח כי העלות התפעולית הממוצעת לכל אינטראקציה נמוכה בסדרי גודל בהשוואה למערכת המסתמכת על עיבוד LLM לכל הודעה נכנסת.

7.6 התפלגות ציוני ודאות (Confidence Distribution)

טבלה 8 - התפלגות סט הבדיקה - confidence

מקור	דיוק בטווח	מהסת %	הודעות	confidence טווח
תוצאות ברורות — rules + LLM	100%	40%	20	(ודאות מלאה) 1.0
בסף המינימלי לחוקים	100%	30%	15	0.85 – 0.99
הודעות עמומות — LLM	91%	22%	11	0.70 – 0.84
הופעל CLARIFY_INTENT	75%*	8%	4	0.50 – 0.69
Metric 1 — Single-Turn	94.0%	100%	50	סה"כ

* הערה לנתוני הדיוק: שיעור ההצלחה של 75% שנמדד ב-Metric 1 (סיווג בתור יחיד) מזנק ל-100% ב-Metric 2 (רמת השיח השלם). הפער נובע מכך שהודעות אשר זוהו תחילה כעמומות והפעילו את מנגנון ה-CLARIFY_INTENT, פוענחו בהצלחה והגיעו לסיווג הנכון בזכות שאלת ההבהרה שיזם הסוכן.

הנתונים המפורטים מעלה מאששים באופן אמפירי את הבחירה בערך הסף (Threshold) של 0.85 כנקודת המעבר בארכיטקטורה. כל ההודעות שחצו רמת ודאות זו (המהוות 70% מסט הבדיקה) סווגו בדיוק מושלם של 100%. ההודעות בטווח הביטחון הבינוני (0.84–0.70) הפגינו רמת דיוק של 91% ירידה מינורית אשר מאשרת כי מודל ה-LLM אכן מתמודד בהצלחה עם הרוב המכריע של מקרי הקצה ("הזנב הארוך" של השפה הטבעית) מבלי להתפשר באופן מהותי על אמינות המערכת.

7.7 דיון: מגבלות מתודולוגיות של הליך ההערכה

למרות התוצאות המבטיחות, קיימות מספר מגבלות מתודולוגיות מהותיות במערך הבדיקה הנוכחי, שראויות לציון ולדיון מפורש:

- גודל מדגם (Sample Size):** גודל המדגם הנבחר (n=50) מעניק אינדיקציה ראשונית חזקה, אך בעל תוקף סטטיסטי מוגבל להסקה על כלל האוכלוסייה. להלן ניתוח מרווח השגיאה:
- רמת סמך (Confidence Level):** 95%
- שגיאה סטטיסטית (Margin of Error):** ~6.6%
- אומדן הדיוק (Accuracy Estimate):** עבור רמת דיוק נצפית של 94%, הדיוק האמיתי באוכלוסייה נאמד בטווח של:

$$94\% \pm 6.6\% \rightarrow [87.4\%, 100\%]$$

משמעות הדבר היא כי הדיוק האמיתי באוכלוסייה נאמד בטווח שבין 87.4% ל-100.6%. עבודות המשך יידרשו לבסס את ההערכה על מדגם רחב יותר (רצוי 200 הודעות ומעלה) על מנת לצמצם את אי-הוודאות הסטטיסטית.

- הטיית בחירה (Selection Bias):** מאחר שסט הנתונים הורכב ונברר על ידי מפתח המערכת, קיים חשש להטיה (Overfitting תפיסתי) כלפי תבניות שיח ואתגרי שפה שנבחנו במהלך תהליך הפיתוח עצמו. על מנת לתקף את

הממצאים בסביבה אותנטית לחלוטין, מומלץ לבצע הערכה משלימה על בסיס יומני שיח (Logs) של משתמשים אמיתיים לאחר פריסת המערכת בייצור (Deployment).

2. **עמימות בקביעת אמת-הקרקע (Ground Truth Ambiguity):** בחלק מההודעות קיימת עמימות סובייקטיבית אינהרנטית בתיוג הידני המקורי. לדוגמה, שיוך המשפט "אני מקווה שכן" לקטגוריית YES או MAYBE נתון לפרשנות. בהעדר הליך מובנה של הסכמה בין-שופטים (Inter-Annotator Agreement) על ידי מעריכים בלתי תלויים, לא ניתן לשלול כי מדד זה היה חושף אי-הסכמה בשיעור של 5%-10% מהתיוגים, מה שהיה משפיע על הערכת ביצועי המודל. זה נובע מאופי וטון השפה אשר לאו דווקא תמיד ניתן לאבחנה.

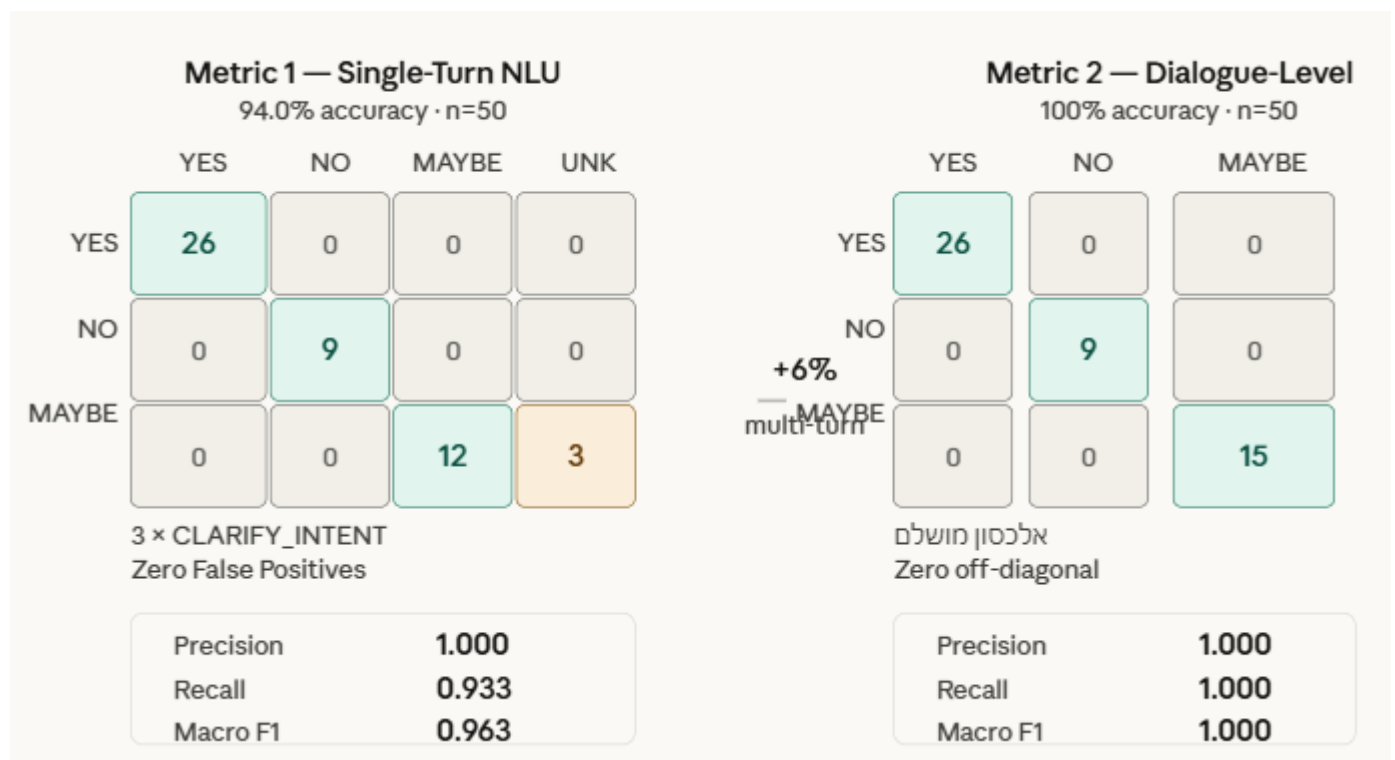
3. **כיסוי לשוני וניבים (Slang & Language):** סט הנתונים אומנם מכסה היטב ביטויי סלנג עברי נפוץ (כדוגמת "בלי נדר", "נשתדל"), אך הוא אינו מייצג בצורה הולמת סלנג אזורי, ניבים של קהילות ספציפיות, או מעברי משלב ושפה (כגון שילוב עברית וערבית). תופעות לשוניות השכיחות בחלקים ניכרים מקהל היעד הפוטנציאלי של המערכת.

על אף המגבלות שנסקרו, תוצאות ההערכה, **הן הכמותיות והן האיכותיות**, מספקות תמיכה אמפירית מוצקה לשתי התזות המרכזיות שעמדו בבסיס פרויקט מחקר זה:

ראשית, הגישה הארכיטקטונית ההיברידית עובדת ומציגה חסינות וביצועים גבוהים (94% דיוק בסיווג תור יחיד, 100% דיוק בסיום האינטראקציה). שנית, למנגנון ניהול השיח הרב-שלבי (Multi-turn) יש תרומה עצמאית, ניתנת לכימות וחיונית לביצועי המערכת (המשתקפת בפער חיובי של 6% למדד הדיוק הכולל ברמת השיח).

7.8 סיכום ממצאי ההערכה

איור 8 - השוואת Confusion Matrix: Single-Turn NLU מול Dialogue-Level



האיור מציג זה לצד זה את שתי מטריצות הבלבול שנמדדו בהערכת המערכת (n=50). בצד שמאל מוצג Metric 1, המודד את ביצועי מנוע ה-NLU הגולמי בתור שיחה בודד: 47 מתוך 50 הודעות סווגו נכון (94.0%), כאשר 3 מקרי UNKNOWN (המודגשים בכתום) אינם נחשבים לשגיאות אלא להפעלה מכוונת של מנגנון הבהרת הכוונה (CLARIFY_INTENT) בשל עמימות בטקסט. בצד ימין מוצג Metric 2, המייצג את הסטטוס הסופי לאחר סיום השיח המלא: כאן מתקבל אלכסון מושלם עם 100% דיוק ואפס חריגות מהאלכסון. החץ המרכזי ממחיש את הפער של +6%. נתון המבטא מספרית את תרומתו הקריטית של מנגנון ניהול השיח הרב-שלבי כרכיב המפצה על אי-ודאות.

איור 9 - ניתוח איכותני: שלוש שיחות מייצגות מיומני המערכת

שיחה	אתגר	מנגנון שהופעל	תוכנה ארכיטקטונית
שיחה 1 MAYBE→YES conf: 0.7→1.0 headcount: 2	כרגע זה פתוח" ← LLM MAYBE (0.7) "נגיע זוג" ← YES exact:2 (1.0)	LLM fallback EffectsPatch rsvpUpdatedAt	חותמת זמן משתנה רק בשינוי סמנטי אמיתי — לא בכל תור שיחה
שיחה 2 YES→NO→YES conf: 0.95 headcount: 2→1	"ברור" → YES AWAITING_HEAD נגיע שניים" → 2 סליחה בסוף" אגיע לבד" → 1	detectChangeIntent היררכיית כוונות SET_RSVP YES+1	"סליחה" = תיקון" ולא ביטול — כלל עסקי מפורש ב decideAction ללא שאלת הבהרה
שיחה 3 NO→YES שגיאת הקלדה "בסוף"	"לא נגיע בסוף" ברור ← NO שינוי, בסוף" "אגיע לבד ← LLM: YES (0.9)	Stateless NLU LLM fuzzy match מעבר קוטבי חלק	— ללא מצב NLU מנתח כל הודעה MongoDB. מחדש מקור האמת = היחיד
EffectsPatch · detectChangeIntent · Stateless NLU: כל שיחה מאמתת רכיב ליבה שונה			

האיור מסכם שלוש שיחות אותנטיות שנבחנו מתוך לוגים של ריצת המערכת (מרץ 2026), הממחישות את יכולות הסוכן בתנאי אי-ודאות. כל תרחיש מציג מורכבות עולה ומפרט את המנגנון הארכיטקטוני שהופעל:

- 1. תרחיש 1 (מנגנון EffectsPatch):** אימות כתיבה מדויקת לבסיס הנתונים רק כאשר מזהה שינוי סמנטי מהותי, למניעת עדכוני סרק.
- 2. תרחיש 2 (זיהוי שינוי כוונה):** יכולת ההבחנה של הסוכן בין "תיקון" לבין "ביטול" (detectChangeIntent) באופן אוטונומי וללא צורך בשאלות הבהרה מיותרות.
- 3. תרחיש 3 (מימוש Stateless):** המחשת העיקרון שבו מנוע ה-NLU מפרש נכון מעבר קוטבי (מ-"לא" ל-"כן") הכולל שגיאות הקלדה. הפרשנות מסתמכת על MongoDB כמקור אמת יחיד (Single Source of Truth) ולא על זיכרון שיחה פנימי פגיע, מה שמבטיח עקביות של הנתונים.

פרק זה הציג הערכה אמפירית דו-צירית של המערכת המיושמת.

- בציר הכמותי:** על בסיס סט בדיקה מבוקר (n=50), הציגה המערכת מדדי דיוק (Precision) מושלמים (1.000) בכל מחלקות הכוונה. נמצא כי דיוק הסיווג בתור שיחה בודד עמד על 94%, וטיפס ל-100% עם השלמת השיח המלא. פער זה, בשיעור של 6%, מכמת באופן ישיר את תרומתו הייחודית של מנגנון ניהול השיח הרב-שלבי כרכיב המפצה באופן אוטונומי על עמימות לשונית.
- בציר האיכותני:** ניתוח שלוש שיחות אותנטיות מיומני המערכת אישש כי הסוכן פועל כשורה דווקא במקרי הקצה שבהם מערכות מבוססות כללים נוטות להיכשל: תיקון כוונה רטרואקטיבי, התגברות על שגיאות הקלדה, וזיהוי מעברים קוטביים בסטטוס ההגעה.

לסיכום, שילוב שתי מתודולוגיות ההערכה מספק תיקוף אמפירי מלא לשתי טענות המחקר המרכזיות: הארכיטקטורה המוצעת הוכחה כיעילה וחסונה לשגיאות, ותרומתו של רכיב ניהול השיח האוטונומי נמצאה כמשמעותית וניתנת למדידה כמותית.

פרק 8: מבנה תוכנה SDLC בעידן ה-AI וה-LLM

פרק זה נכתב בשקיפות מלאה, בהתאם להנחיות המנחה פרופ' אהוד גודס, ומפרט את תהליך הפיתוח בפועל, מי תכנן, מה נוצר עצמאית, ובאיזה אופן כלי AI שימשו כעוזרים בתהליך.

8.1 תהליך התכנון

כל ההחלטות הארכיטקטוניות והתכנוניות של הפרויקט התקבלו על ידי כותב עבודה זה באופן עצמאי. בשלב הראשוני, לפני שנכתבה שורת קוד, הוגדרו:

המבנה הכללי של המערכת כ-Monorepo עם הפרדה בין bot-service ל-admin-web, בחירת הסטאק הטכנולוגי (Node.js, TypeScript, Telegraf, MongoDB, Vue 3), תכנון מודל הנתונים עם שלוש קולקציות (Campaign, Guest, Invite) וההבחנה בין rsvpUpdatedAt ל-lastResponseAt, ועיצוב לוגיקת ה-RSVP הכוללת את ה-14 שלבי חילוץ הכמות, היררכיית אותות זיהוי השינוי, ומנגנון ה-EffectsPatch.

בתהליך התכנון נעשה שימוש בשיח עם מנועי צ'אט כדוגמת GPT כדי לאמת רעיונות, לבחון חלופות ארכיטקטוניות, ולחדד ניסוחים. שיח זה שימש כ"שותף חשיבה" לבדיקת ההגיון ולא כמקור להחלטות עצמן. כל בחירה מהותית להפרדת שכבות ה-Adapters & Ports, דרך ה-confidence threshold של 0.85, ועד מדיניות ה-"never guess", נובעת מהבנה עצמית של הבעיה.

8.2 מסגרת מתודולוגית: פיתוח אינקרמנטלי בסביבת מפתח יחיד

תהליך הפיתוח של הפרויקט התבסס על עקרונות ה-Incremental and Iterative Development. בגישה זו, במקום ניסיון לבנות מערכת שלמה בצעד אחד, כל איטרציה (Iteration) מייצרת מערכת פונקציונלית עובדת ורק לאחר שהיא מאומתת, מתווספת עליה יכולת חדשה.

הבחירה במתודולוגיה זו נבעה משני שיקולים מרכזיים:

- ניהול אי-ודאות טכנולוגית:** הבנייה של מנוע חוקים מבוסס AI טומנת בחובה אי-ודאות מובנית. לא ניתן לתכנן מראש את גבולות היכולת של מנוע ההחלטות לפני שבונים ומודדים אותו בפועל. לכן, כל סבב פיתוח החל בשאלת מחקר והסתיים בנתון מדיד שהכתיב את הצעדים בגלל הפיתוח הבא.
 - אופטימיזציה למפתח יחיד (Solo Developer):** במערך משאבים של מפתח בודד, הגישה האינקרמנטלית חיונית לניהול מורכבות. היא מאפשרת מיקוד גבוה ביחידה תפקודית אחת בכל פעם, במקום פיצול קשב בין רכיבים רבים. עבודה זו צמצמה את ה-"Overhead" הניהולי והבטיחה שכל רכיב ב-LangGraph נבחן לעומק מול יכולות ה-NLU הקיימות לפני שהמערכת הורחבה לתרחישים נוספים.
- בדרך זו, הפרויקט התקדם בצורה של "שכבות" יציבות, כאשר כל שכבה מהווה תשתית בטוחה ליכולות האוטונומיות שנוספו מעליה ב-decideAction.

8.3 אבולוציית הפיתוח - ארבעה גלים

הפרויקט עבר ארבע גרסאות מרכזיות, שכל אחת הוסיפה שכבת יכולת על גבי הקודמת:

- גל ראשון: מנוע חוקים טהור (Rule-Based Only)**
הגרסה הראשונה של הסוכן פעלה ללא כל מעורבות של LLM. מנוע החוקים (rules.ts) כולל הנירמול, זיהוי הכוונות, 14 שלבי חילוץ הכמות ו-Levenshtein fuzzy matching, פותח ונוסה בשלמותו לפני שנשקלה כלל הוספת LLM. הרציונל: לוודא שהבסיס הדטרמיניסטי איתן לפני שמוסיפים מרכיב הסתברותי. גרסה זו הדגימה יכולת טובה לתרחישים נפוצים וסיפקה קו בסיס ברור לאמידת התרומה של ה-LLM לאחר מכן.
- גל שני: שילוב LLM ויצירת תגובות אנושיות**
רק לאחר שהגרסה הראשונה הניבה תוצאות מספקות עבור הודעות מסוג "כן מגיע", "לא יכול", ו"אולי", הוחלט להוסיף את מרכיב ה-LLM. השילוב נועד לשתי מטרות: הרחבת יכולת הפענוח לזנב הארוך של הודעות עמומות שמנוע החוקים לא הצליח לסווג בביטחון, ויצירת תגובות אנושיות, מגוונות ומותאמות הקשר במקום תבניות קבועות. כחלק מגל זה מומשה שכבת ה-Prompt Engineering המפורטת בפרק 5, כולל Five-Shot Examples וכלל ה-"never guess".

• גל שלישי: מעבר ל-LangGraph וניהול מצב מתקדם

הגרסה הראשונה והשנייה ניהלו את השיחה דרך קובץ `rsvpFlow.ts`, לוגיקה פרוצדורלית שטוחה. כאשר התברר שנדרשת גמישות גדולה יותר, בפרט לטיפול בשינויי כוונה בתוך לולאת שאלת הכמות, ולצורך ניהול הקשר מורכב לאורך תורות מרובות. לכן הוחלט להשקיע בכניסה לספריית `LangGraph`. ההחלטה הייתה גם הנדסית (יכולות הרחבה, `testability, observability`) וגם אקדמית לצורך לימוד מעמיק של הכלי המוביל בתחום ה-`Agentic AI Workflows`. המעבר התברר כהצלחה: `decideAction` כפונקציה טהורה עם 19 בדיקות יחידה, ה-`fast path` ב-`interpretHeadcount`, והיכולת להוסיף צמתים ללא שבירת הניתוב הקיים. כולם יתרונות ישירים של הגישה החדשה. קובץ `rsvpFlow.ts` הישן נשמר ב-`codebase` כ-`legacy reference` אך אינו מיובא עוד.

• גל רביעי: הקשחת ממשקים (TypeScript Strict Interfaces)

לאחר שהתקבלה גרסה יציבה יחסית, בוצע סיבוב של `hardening`: בדיקות יחידה, כל ממשקי ה-`TypeScript` עודכנו להיות קשיחים יותר `Zod schema`, `discriminated unions` בגבולות ה-API ופענוח של תגובות ה-LLM, ואכיפת `strict mode` לאורך כל `codebase`. שלב זה הפחית משמעותית את מרחב הבאגים האפשריים בגבולות שכבות.

8.4 כלי AI בתהליך הפיתוח (Coding Assistance)

כל הקוד נכתב בסביבת `editor-Cursor` מבוסס `VS Code` המשלב מנוע השלמה אוטומטית מבוסס `LLM`. `Cursor` שימש לכתובת `boilerplate`, השלמת פונקציות, ורפקטורינג מהיר תחת הכוונה מלאה של המפתח. הכותב סיפק את הסכמות, ה-`interfaces`, ואת הלוגיקה הרצויה. `Cursor` תרם ביצוע מהיר ועקבי של כוונות אלו. התייעוד הטכני של הפרויקט (10 קבצי `docs`, `AGENTS.md`, `CLAUDE.md`) נוצר בעזרת `Claude Code` כלי שורת פקודה של `Anthropic` לתייעוד ולניתוח קוד. כמו כן, כתיבת הדוח השוטפת נוהלה מול מספר מודלים (`GPT`, `Claude`), כשהמפתח מנהל את הכיוון, מספק את התוכן המהותי, ומשתמש בכלים לצורך ניסוח וסגנון.

8.5 שימוש ב-LLM כחלק מריצת המערכת (Runtime AI)

בנוסף לשימוש ב-AI כעוזר פיתוח, המערכת עצמה משלבת LLM בזמן ריצה בשני תפקידים:
LLM לפענוח NLU: `Claude 3 Haiku` מופעל כ-`Fallback` כאשר מנוע החוקים מחזיר `confidence` מתחת ל-0.85. הוא מקבל `System Prompt` עם `Five-Shot examples` ומחזיר `JSON` מובנה.
LLM ליצירת תגובות (אופציונלי): כאשר הדגל `RSVP_USE_LLM_RESPONSES=true` מופעל, `Claude` מייצר תגובות טבעיות ומותאמות הקשר במקום תבניות קבועות. ברירת המחדל היא `false` - תבניות מהירות וחינמיות, כיוון שרוב האינטראקציות אינן דורשות יצירתיות לשונית. בנוסף זה מאפשר להריץ את הסוכן גם ללא `API Key` מ-`provider` למודל של `Claude 3 Haiku`, בכך ניתן להשתמש בה גם ללא עלות צריכת טוקנים למקרה שיש רצון כזה.

8.6 ניהול ידע והנחיית סוכנים (Agentic Instruction Layers)

בפרויקט זה, התייעוד אינו משמש רק ככלי עזר אנושי, אלא מהווה רכיב קריטי בארכיטקטורת הפיתוח. השימוש בקבצי הנחיה ייעודיים כגון `CLAUDE.md` ו-`AGENTS.md` מתבסס על עקרונות של **Context Engineering** – אספקת "מצפן" לוגי לסוכן ה-AI בתחילת כל סשן עבודה.

א. תפקיד קבצי ההנחיה כ-System Prompt מורחב

כלי פיתוח מודרניים (כגון `Claude Code` או `Cursor`) סורקים באופן אוטונומי את שורש הפרויקט (`Root`) ומחפשים קבצי הוראות. קבצים אלו מוזרקים לתוך ה-**Context Window** של המודל בכל פנייה, ובכך הם משמשים כ-`High-level` `System Prompt` המגדיר את ה"אני מאמין" של הפרויקט עוד לפני כתיבת שורת הקוד הראשונה.

ב. שימור הארכיטקטורה ומניעת "סחף לוגי" (Architectural Drift)

אחד האתגרים בעבודה עם LLMs הוא הנטייה שלהם להציע פתרונות גנריים שעלולים לסתור את עקרונות הפרויקט. קבצי ההנחיה משמשים כ"שומר סף" (`Guardrails`):

- **אכיפת ה-Hexagonal Architecture:** המודל מקבל הנחיה מפורשת האוסרת על ייבוא אדפטרים (כגון קריאות ל-`MongoDB`) לתוך שכבת ה-`Domain`.

- **דוגמה להנחיה מתוך: AGENTS.md**

"Always maintain strict separation between NLU logic and Infrastructure. Do not reference 'mongo' or 'telegraf' inside the rsvpFlow.ts file. Use ports and interfaces only."

- **ניהול תלויות:** הגדרת חוקי ברזל המונעים מהסוכן להוסיף ספריות חיצוניות ללא אישור, ובכך שומרים על ה-Codebase רזה ומדויק.

- ג. **עקביות סגנונית ו-Pattern Matching**

באמצעות הגדרה מראש של תבניות (Patterns), הבטחנו שכל קוד שיוצר על ידי ה-AI יעמוד בסטנדרטים האקדמיים וההנדסיים של הפרויקט:

- **Typescript Patterns:** הנחיה לשימוש ב-**Discriminated Unions** לניהול מצבי RSVP, מה שמבטיח Type-Safety מקסימלי.

- **מבנה בדיקות (Testing Consistency):** הגדרת תבנית אחידה עבור Vitest המבטיחה כי כל פונקציה חדשה שהסוכן מייצר תלווה בבדיקות יחידה תואמות, תוך שמירה על כיסוי קוד (Coverage) גבוה.

- ד. **תיעוד אוטונומי וסנכרון (Live Documentation)**

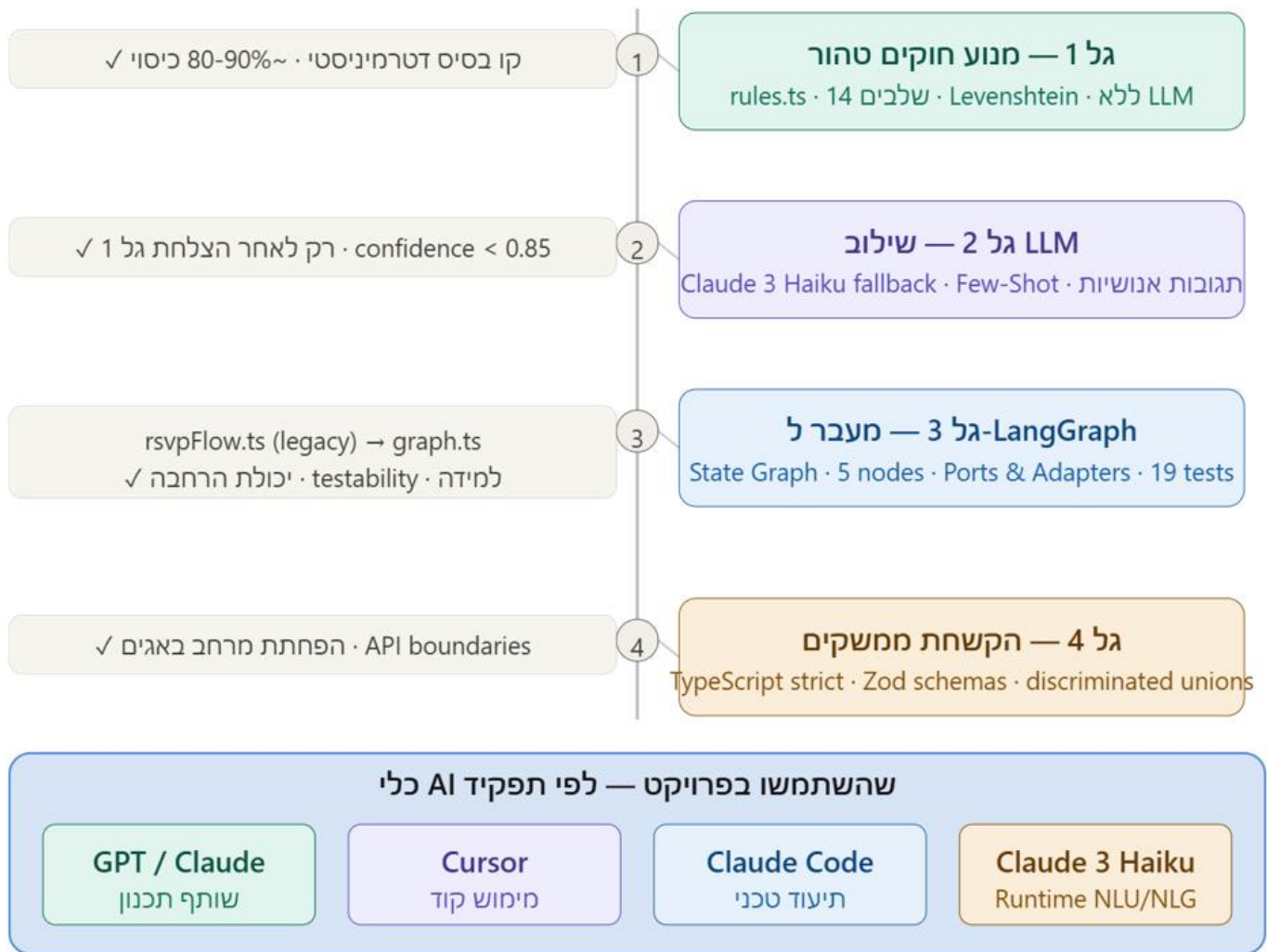
השימוש בסוכנים מאפשר תהליך של **תיעוד חוזר (Feedback Loop)**: הסוכן סורק את השינויים שביצע בקוד ומעדכן באופן אוטונומי את קבצי התיעוד בתיקיית docs/. פעולה זו מבטיחה כי התיעוד האקדמי המוגש בדוח זה משקף בדיוק של 100% את המימוש בפועל, ללא פערים הנובעים משכחה או עדכוני גרסאות.

ניהול הידע באמצעות קבצי ה-Agent Instructions הפך את ה-AI משותף לכתיבת קוד (Code Assistant) למנהל פרויקט טכני (Technical Lead) שמכיר את עקרונות העיצוב לעומקם ומבטיח את איכות התוצרים לאורך זמן.

8.7 הנדסת הקשר (Context Engineering) בתהליך הפיתוח

נבחר להדגיש כי תפקידו של כותב העבודה השתנה מ"כותב קוד" מסורתי ל-**Context Engineer**. במקום לבקש מה-AI "לכתוב פונקציה", התהליך התמקד בבניית ההקשר המדויק עבור המודל:

1. **Selection of Context:** בחירה ידנית ומדויקת של קבצי המקור (Interfaces, Schemas) הרלוונטיים למשימה, כדי למנוע מהמודל "הזיות" הנובעות מהקשר חסר או עודף.
 2. **Schema-First Development:** הגדרת ה-Contract (הממשק) ב-TypeScript באופן ידני לחלוטין לפני הפעלת ה-AI. כאשר המודל מוגבל על ידי Interface קשיח, הקוד שהוא מייצר חייב לעמוד בדרישות הטיפוסים, מה שמבטיח אינטגרציה חלקה.
 3. **Iterative Refinement:** עבודה בסבבים קצרים שבהם המפתח בוחן את הפלט של ה-AI, מזהה כשלי לוגיקה ברמת ה-High-level, ומעדכן את ההנחיות ב-CLAUDE.md כדי למנוע הישנות של טעויות אלו.
- השילוב בין החלטות ארכיטקטוניות עצמאיות לבין שימוש מושכל ב-Context Engineering וסוכני קוד, איפשר להגיע לרמת הקשחה (Hardening) ודיוק גבוהים בזמן קצר, תוך שמירה על סטנדרטים הנדסיים של מערכות Production מורכבות.



4 גלים - כל גל בנה על הקודם: גל 1 ביסס קו בסיס דטרמיניסטי, גל 2 הוסיף LLM רק לאחר הוכחת יכולת, גל 3 העביר לניהול גרף מצבים, גל 4 הקשיח את הממשקים. תיבת הכלים בתחתית מפרידה בין 4 תפקידים שונים של פרויקט - תכנון, מימוש, תיעוד, וריצה.

פרק 9: דיון וסיכום

9.1 סיכום ההישגים

בתחילת הפרויקט ניסחתי שאלת מחקר שנראתה, על פניה, כמעט בנאלית: האם ניתן לבנות בוט שיענה על שאלה אחת פשוטה - "האם אתה מגיע לאירוע?" כשהתחלתי לחקור לעומק, הבנתי שמאחורי השאלה הפשוטה מסתתרת בעיה עשירה ומרתקת של הבנת שפה, ניהול הקשר, קבלת החלטות בזמן אמת, ואמינות הנדסית.

המערכת שנבנתה, EZ-Event-BOT, היא פתרון קצה-לקצה פועל: אורחים מקבלים קישור אישי לאפליקציית טלגרם, מנהלים שיחה חופשית בעברית, והמארח רואה את התוצאות בזמן אמת בלוח בקרה ויזואלי. אין טפסים, אין חיכוך, ואין התערבות אנושית בתהליך. מה שנתפס בהתחלה כבעיה לוגיסטית שגרית של ניהול אישורי הגעה הפך לזירת מחקר מאתגרת שנגעה בעיבוד שפה עברית, ארכיטקטורת סוכנים, ניהול מצב מבוזר, ואינטגרציה של מודלי שפה גדולים בתוך מערכת תוכנה אמינה.

ההישג המרכזי אינו הבוט עצמו, אלא ההוכחה שהגישה ההיברידית עובדת. כ-80-90% מהמסרים מטופלים על ידי מנוע חוקים דטרמיניסטי, מהיר וחסכוני. רק הזנב הארוך של הודעות עמומות מגיע ל-LLM. שלוש השיחות שנבחנו בפרק 7 הסתיימו כולן בסיווג מדויק הכולל מקרים מורכבים של שינוי כוונה ("בסוף אגיע לבד" לאחר "אני אהיה עסוק") ותיקון כמות רב-שלבי אסינכרוני מורכב שומר הקשר משיחות עבר.

9.2 שלוש תובנות ארכיטקטוניות שנצברו

לאורך החודשים שבהם עבדתי על הפרויקט, שלוש תובנות עלו שוב ושוב. בכל פעם שנתקלתי בבאג, בפגישת תכנון, ובסיקור הקוד.

- **תובנה ראשונה: גרף מצבים הוא לא "לימוד" אלא חובה לתכנון workflow agentic.** ניהלתי את השיחות תחילה דרך לוגיקה פרוצדורלית ב-rsvpFlow.ts. הקוד עבד. אבל ברגע שנוספו מקרי הקצה כמו מה קורה אם אורח בתוך לולאת שאלת הכמות פתאום אומר "רגע, בסוף לא מגיע"? ה-FSM החל להיסדק. כל מקרה קצה חדש דרש קשת נוספת, ולאט-לאט הקוד הפך ל-spaghetti. המעבר ל-LangGraph לא היה על-מנת לעקוב אחרי טרנד טכנולוגי אלא הוא פתר בעיה אמיתית שחוויתי. decideAction כפונקציה טהורה עם בדיקות יחידה, בלי async ובלי side-effects מצד ה-LLM (בסבירות גבוהה), היא הישג אדריכלי שאני גאה בו.
- **תובנה שנייה: הפרדת שכבות היא לא בירוקרטיה - היא חסכון זמן לפרויקט מתמשך.** כשהוחלפו תבניות NLG ברירת המחדל בתגובות מבוססות LLM, הדומיין לא נגע בשינוי. כשהוחלט לנסות ראשית עם מנוע חוקים בלבד ורק לאחר מכן לשלב LLM, ה-NluPort סיפק הפשטה מושלמת. ה-AnthropicAdapter פשוט הוחלף ב-stub. עבודה זו לימדה אותי שהממשקים אינם "overhead טכנוני", אלא הם מה שמאפשר להכניס ידיים לתוך מערכת חיה מבלי לשבור אותה.
- **תובנה שלישית: LLM בתוך מערכת אמינה הוא אתגר הנדסי בפני עצמו.** לפני הפרויקט חשבתי שהאתגר הוא "לגרום ל-LLM לענות נכון". בפועל, האתגר היה לגרום למערכת שסומכת על LLM לפעול בצורה ודאית. Zod, ולידציה, graceful degradation, regex fallback, UNKNOWN, למניעת new Date בתוך הדומיין, כל אלה נוצרו לא מתוך ניקיון יתר, אלא מתוך פגיעה ישירה בבאגים שנגרמו מהעדר שכבות הגנה אלו.

9.3 שינוי פרדיגמה: מהנדס תוכנה בעידן ה-AI

אחד הדברים שהכי אהבתי בפרויקט הזה הוא עד כמה המאמץ ההנדסי שינה אופי. כשמהנדס כותב קוד מסורתי, הוא שואל: "מה הלוגיקה הנכונה?" ומקודד אותה. כשמהנדס בונה מערכת מבוססת LLM, השאלה הופכת ל: "מה ההתנהגות הנכונה?" - שאלה שונה לגמרי.

ה-Prompt Engineering שביצעתי לא היה "כתיבת הוראות לבוט". זה היה תכנון מערכת. כל Few-Shot Example ב-System Prompt הוא באמצעות test case חי שמכוון את המודל לאזורי החלטה ספציפיים. כלל ה-"never guess" הוא מדיניות עסקית מקודדת בשפה טבעית. הגבלת 200 הטוקנים היא שיקול ארכיטקטוני של עלות מול ביצועים. המעבר הזה ממחשבה אלגוריתמית למחשבה התנהגותית, הוא לדעתי אחד השינויים המהותיים ביותר בהנדסת תוכנה בעשור הנוכחי. המהנדסים שיצליחו הם לא אלה שיחליפו קוד ב-Prompts, אלא אלה שיבינו מתי לקודד, מתי להנחות, ומתי לשלב.

9.4 כיווני המשך ומחקר עתידי

- המערכת במצבה הנוכחי היא גרסה 1.0 - פועלת, נבדקת, ובעלת ארכיטקטורה שמוכנה לגדול. שלושה כיוונים מחקריים נראים לי כמשמעותיים במיוחד:
- **הרחבה לדומינים נוספים.** הארכיטקטורה של הסוכן, הפרדת Ports, גרף EffectsPatch, LangGraph אינה ספציפית ל-אישורי הגעה (RSVP). בשינוי קטן יחסית של decideAction ושל מנוע החוקים, אפשר להסב את המערכת לניהול תורים במרפאה, אישורי השתתפות בהתנדבות, או קבלת הזמנות לאירועי עסקים. מחקר עתידי יכול לבחון כמה "קשיח" הדומיין הנוכחי. כלומר, איזה אחוז מהלוגיקה הוא RSVP-specific ואיזה אחוז הוא generic dialogue management.
 - **מערכת רב-סוכנים.** כיום הסוכן מנהל שיחה מול אורח בלבד. ניתן להוסיף סוכן מקביל שיפעל מול המארח ויזהר אוטומטית שהאולם מתמלא, יציע לסגור הרשמות, או יתריע על חריגות (כגון 20 אורחים שהצהירו על דרישות תזונה מיוחדות). מודל Multi-Agent עם LangGraph ישמש כפלטפורמה טבעית לתיאום בין הסוכנים.
 - **עיבוד קולי.** הזרמת הודעות קוליות מ-Telegram לצינור NLU הקיים, עם שכבת Speech-to-Text בניהם. היא הרחבה ישירה שדורשת שינוי מינימלי בארכיטקטורה. הפרדת שכבות Ports מבטיחה שההודעה הקולית המתומללת תיכנס ל-interpretMessage בדיוק כמו הודעת טקסט, ללא כל שינוי בלוגיקת הדומיין.

סיכום ומבט לעתיד

כשאני מסתכל לאחור על הפרויקט הזה בכללותו, מה שנשאר חקוק בזיכרוני אינם הבאגים שתוקנו או שורות הקוד שנכתבו, אלא הרגע שבו הכל התחבר לכדי יחידה אחת חכמה. עבורי, המניע המרכזי והעניין העמוק לאורך כל הדרך היה חקירת

תחום ה-**Agentic Flow**. היכולת לקחת את ה-LLM ולהפוך אותו מרכיב שפה גנרי למנוע שמניע מערכת תוכנה מורכבת, אוטונומית וברת-ביצוע.

הלמידה של עקרונות הסוכנות (Agency) והתכנון של זרימת העבודה (Flow) הם שדחפו אותי לפצח איך משלבים בינה מלאכותית בתוך לוגיקה עסקית קשיחה. רגע השיא היה כששלחתי לבוט הודעה פשוטה ויומיומית: **"נגיע זוג בסוף"**, וקיבלתי בחזרה: **"תודה! נרשמת, 2 אנשים"**.

מאחורי התשובה הקצרה הזו מסתתרות שכבות שלמות שעמלתי עליהן: נירמול טקסט חופשי, חילוץ ישויות וכמויות, ניהול מצב (State Management) מורכב ועדכון דינמי של בסיס נתונים. זהו רגע שקוף לחלוטין למשתמש, אך עמוק ורב-שכבתי מבחינה הנדסית.

זהו, בסופו של דבר, היעד אליו כיוונתי: לקחת משימה אנושית פשוטה, שבאופן מסורתי גוזלת זמן ומשאבים, ולהפוך אותה לאוטומציה אלגנטית בעידן ה-**Agentic AI**. השאיפה שלי הייתה ליישם אלגוריתמיקה שמותאמת לעבודה עם סוכנים חכמים, ולהפוך מורכבות הנדסית גדולה לפשטות חווייתית – **"Complexity presented as simplicity"**.

איור 12 מסכם את שלושת ממדי הדיון: התובנות הארכיטקטוניות שנצברו (סעיף 9.2), שינוי הפרדיגמה בעבודת המהנדס (סעיף 9.3), וכיווני המחקר העתידי

איור 12 - תובנות, שינוי פרדיגמה, וכיווני עתיד



- **השורה הראשונה** מציגה את שלוש התובנות הארכיטקטוניות שנדונו בסעיף 9.2: גרף מצבים כהכרח לניהול workflow אגנטי (ולא בחירה סגנונית), הפרדת שכבות כחיסכון זמן אמיתי לאורך מחזור חיי הפרויקט, ו LLM-בתוך מערכת אמינה כאתגר הנדסי עצמאי הדורש Guardrails, Zod ו-graceful degradation.
 - **השורה השנייה** ממחישה את שינוי הפרדיגמה שתואר בסעיף 9.3: המעבר מהנדסה מסורתית ("מה הלוגיקה?") להנדסת AI ("מה ההתנהגות?"). הממשק בין שתי העמודות - Prompt Engineering, Few-Shot, Guardrails - הוא הגשר שמהנדסי הדור הבא יצטרכו לשלוט בו.
 - **השורה השלישית** מציגה את שלושת כיווני המחקר העתידי שנדונו בסעיף 9.4: הכללת הדומיין לתחומים נוספים (רפואה, התנדבות, עסקים), ארכיטקטורת Multi-Agent עם Organizer Agent לתיאום בין סוכנים, ועיבוד קולי עם Speech-to-Text ללא שינוי בצינור ה-NLU הקיים.
- אלו מסכמים את הרגע המוכיח של הפרויקט כולו: ההודעה "נגיע זוג בסוף" והתגובה "תודה! נרשמת, 2 אנשים" - שקוף למשתמש, אך מאחוריו נירמול, חילוץ כמות, ניהול מצב ועדכון בסיס נתונים. המורכבות שתוצג כפשטות.

עיבוד שפה טבעית ומודלי שפה

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. <https://arxiv.org/abs/1810.04805>
- LeCun, Y. (2022). *A path towards autonomous machine intelligence* (Version 0.9.2). OpenReview. <https://openreview.net/pdf?id=BZ5a1r-kVsf>
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- Marcus, G. (2020). *The next decade in AI: Four steps towards robust artificial intelligence*. arXiv preprint arXiv:2002.06177. <https://arxiv.org/abs/2002.06177>
- Seker, A., Bandel, E., Bareket, D., Brusilovsky, I., Greenfeld, R. S., & Tsarfaty, R. (2021). AlephBERT: A Hebrew large pre-trained language model to start-off your Hebrew NLP application with. *arXiv preprint arXiv:2104.04052*. <https://arxiv.org/abs/2104.04052>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://arxiv.org/abs/1706.03762>

סוכנים אוטונומיים וניהול שיח

- Adiwardana, D., Luong, M. T., So, D. R., Hall, J., Fiedel, N., Thoppilan, R., ... & Le, Q. V. (2020). Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977*. <https://arxiv.org/abs/2001.09977>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing* (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*. <https://arxiv.org/abs/2210.03629>

ארכיטקטורת תוכנה

- Cockburn, A. (2005). *Hexagonal architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

כלים וספריות

- Anthropic. (2024). *Claude 3 model family: Technical overview*. <https://www.anthropic.com/news/claude-3-family>
- LangChain AI. (2024). *LangGraph: Build stateful, multi-actor applications with LLMs* [Software library]. <https://github.com/langchain-ai/langgraph>

תשתיות ובסיסי נתונים

קוד המקור

Heiblum, I. (2026). *EZ-Event-BOT: Conversational RSVP agent for events via Telegram* [Source code].
<https://github.com/Srul123/EZ-Event-BOT>

נספח: סכמת בסיס הנתונים

1.1 סקירת מודל הנתונים ועקרונות התכנון

המערכת משתמשת ב-MongoDB כבסיס נתונים, מנוהל דרך Mongoose ODM. הבחירה ב-MongoDB נבעה משלושה שיקולים: גמישות הסכמה לשדות אופציונליים שמשתנים לאורך מחזור חיי ה-RSVP (כגון headcount שנוסף רק לאחר אישור), מהירות כתיבה גבוהה לעדכונים פרקטיים (set\$ דליל על מסמכים קיימים), והתאמה טבעית לייצוג מצב שיחה כמסמך אחד מאוחד.

המודל כולל שלוש קולקציות בלבד של Guest, Campaign, ו-Invite, המיישמות שתי קשרים One-to-Many. עיצוב זה מביא לידי ביטוי שלושה עקרונות מנחים:

- **עיקרון 1: הפרדה בין ישויות לוגיות.** ה-Guest מנותק מה-Campaign. בכך שמירה על עקרון ה-Single Responsibility: Campaign מייצג את האירוע, Guest מייצג את מצב האורח הספציפי. פרידה זו מאפשרת שאילתות יעילות כמו שליפת כל אורחי קמפיין היא פשוט ({ campaignId } find עם אינדקס).
- **עיקרון 2: Invite כשכבת זיהוי עצמאית.** ה-Invite אינו "הזמנה" לוגית במובן העסקי, הוא טוקן זיהוי שמאפשר לקשר בין Telegram user לאורח ב-MongoDB. ניתוק שכבת הזיהוי מה-Guest מאפשר יצירת קישורים מחדש (re-generate) מבלי לפגוע בנתוני ה-RSVP כך שכל הטוקנים הישנים נשמרים תקפים.
- **עיקרון 3: denormalization מכוון.** השדה campaignId מופיע הן ב-Guest הן ב-Invite, למרות שניתן היה לגזור אותו מ-Guest.campaignId → guestId. זוהי החלטה מכוונת: היא מונעת join בזמן שליפת אורח מטוקן, ומאפשרת שאילתת ({ campaignId } find Invite ישירה ללא lookup נוסף.

1.2 Collections ותיאור השדות

1 - Campaign Model Document

מייצגת אירוע שיצר המארח. כל קמפיין הוא שורש האחריות (aggregate root) עבור אורחים וקישורים.

תיאור ונימוק				ברירת מחדל		נדרש טיפוס		שדה
מפתח ראשי של MongoDB				אוטו	אוטו	אוטו	ObjectId	_id
שם פנימי לצרכי המארח, אינו מוצג לאורחים				—	—	כן	String	name
כותרת האירוע הציבורית — מוצגת בפתיחת שיחת הבוט				—	—	כן	String	eventTitle
תאריך תצוגה חופשי (למשל "שישי, 25 ביולי"). מאוחסן כ-String ולא כ-Date				—	—	לא	String	eventDate
מכיוון שאין צורך בחישובי תאריך, ומבנה חופשי מאפשר פורמטים עבריים גמישים				—	—	כן	Date	scheduledAt
תאריך ושעה לתחילת הקמפיין. מאוחסן כ-Date אמיתי לצורך שאילתות scheduler עתידיות (\$lte: now)				—	—	כן	Date	scheduledAt
מחזור חיי הקמפיין : DRAFT → SCHEDULED → RUNNING → COMPLETED / FAILED				—	—	לא	Enum	status
Mongoose timestamp				אוטו	אוטו	אוטו	Date	createdAt
Mongoose timestamp				אוטו	אוטו	אוטו	Date	updatedAt

אינדקס: { scheduledAt: 1, status: 1 } - אינדקס מורכב לשאילתת scheduler עתידית: "מצא קמפיינים שה-scheduledAt שלהם עבר וסטטוסים SCHEDULED". האינדקס המורכב יעיל פי כמה מסינון בשני שלבים נפרדים.

Document Model Guest – 2

לב המערכת. מסמך Guest הוא מקור האמת (Source of Truth) לכל מה שנוגע למצב ה-RSVP של אורח ספציפי כולל מצב השיחה עם הבוט.

שדה	נדרש	טיפוס	ברירת מחדל	תיאור ונימוק
_id	אובייקט	ObjectId	אובייקט	מפתח ראשי
campaignId	כן	ObjectId (FK)	—	מפנה ל Campaign._id - אינדקס ללא ייחודיות - אורח אחד לקמפיין אחד, אך ייתכנו אלפי אורחים לקמפיין
name	כן	String	—	שם מלא של האורח. מוצג בפתחה ובתגובות הבוט
phone	כן	String	—	מספר טלפון. מאוחסן כמחרוזת חופשית לתמיכה בפורמטים שונים. אינדקסי אך לא ייחודי - אותו טלפון יכול להופיע בקמפיינים שונים
rsvpStatus	לא	Enum	'NO_RESPONSE'	ממד מצב 1: סטטוס ה-RSVP-הלוגי. ארבעה ערכים: NO_RESPONSE, YES, NO, MAYBE
headcount	לא	Number	null	מספר המשתתפים = null. לא סופק עדיין (שונה מ-0). ולידציה min: 0:
conversationState	לא	Enum	'DEFAULT'	ממד מצב 2: שלב השיחה = DEFAULT. מצב רגיל; YES_AWAITING_HEADCOUNT = ממתינ לכמות. מוצמד ל DB-לא רק ל (Session-להבטחת עמידות לאתחול שרת
rsvpUpdatedAt	לא	Date	null	מתעדכן אך ורק כאשר rsvpStatus או headcount השתנו בפועל. מאפשר למארח לראות "מתי חלה שינוי בנתון" בנבדל מ-"מתי האורח כתב משהו"
lastResponseAt	לא	Date	null	מתעדכן בכל הודעה שעובד הבוט, כולל ACK_NO_CHANGE מייצג "מתי האורח היה פעיל לאחרונה"
clarificationAttempts	לא	Number	0	סופר ניסיונות שאלת הבהרה על כמות. עד 3 ניסיונות — לאחר מכן STOP_WAITING
lastClarificationReason	לא	Enum	null	סיבת העמימות האחרונה (FAMILY_TERM, RANGE_OR_APPROX) (לשיפור שאלת ההבהרה הבאה
createdAt	אובייקט	Date	אובייקט	Mongoose timestamp
updatedAt	אובייקט	Date	אובייקט	Mongoose timestamp

אינדקסים: { campaignId: 1 } - השאילתה הנפוצה ביותר: "תן לי את כל האורחים של קמפיין X". { phone: 1 } - לשימוש עתידי, למשל מניעת כפילויות או שיוך חוצה-קמפיינים.

הבחנה סמנטית קריטית בין rsvpUpdatedAt ל-lastResponseAt: זוהי אחת מהחלטות התכנון המהותיות ביותר בסכמה. שני שדות הזמן נועדו למידע שונה לגמרי. lastResponseAt עונה על השאלה "האם האורח בכלל רואה את ההזמנה ומגיב?". rsvpUpdatedAt עונה על השאלה "מתי לאחרונה חלה שינוי לוגיסטי בנתוני ה-RSVP?". ניתוח ב-Admin Dashboard המבלבל בין השניים יוביל למסקנות שגויות כמו למשל, לראות שאורח "עדכן" ב-17:23 כשבפועל הוא רק שלח "שלום".

Document Model Invite – 3

קולקציה דקה שתפקידה בלעדי הוא מיפוי טוקן ↔ אורח. היא שכבת הזיהוי של המערכת.

שדה	נדרש	טיפוס	ייחודי	תיאור ונימוק
_id	אובייקט	ObjectId	—	מפתח ראשי

תיאור ונימוק	ייחודי	נדרש	טיפוס	שדה
inv_ + 12 בתווי base64url שנוצרו ב. crypto.randomBytes(8)-האינדקס הייחודי מבטיח lookup O(1) ומונע כפילויות	כן	כן	String	token
מפנה ל-Guest._id-	לא	כן	ObjectId (FK)	guestId
מפנה ל denormalized — Campaign._id-במכוון למניעתjoin	לא	כן	ObjectId (FK)	campaignId
מוגדר בפעם הראשונה שהאורח לחץ על הקישור. מאפשר מעקב "מי פתח את ההזמנה"	—	לא	Date	usedAt
Mongoose timestamp	—	אופט	Date	createdAt
Mongoose timestamp	—	אופט	Date	updatedAt

אינדקסים: { token: 1 } - ייחודי - primary access pattern. { guestId: 1 } - בדיקה אם כבר נוצרו קישורים לאורח. { campaignId: 1 } - ספירת/שליפת כל הקישורים לקמפיין.

7.3 קשרים בין ישויות וזרימת הנתונים

זרימת הנתונים במערכת ניתנת לחלוקה לארבעה שלבי מחזור חיים:

שלב יצירה: POST /api/campaigns יוצר Campaign document אחד ו-N Guest documents (bulk insert) בפעולה אטומית. כל Guest מתחיל עם rsvpStatus: NO_RESPONSE.

שלב הפצה: POST /api/campaigns/:id/generate-telegram-links יוצר N Invite documents, אחד לכל Guest. הטוקן נשמר עם אינדקס ייחודי. קישורים ישנים (מ-generate קודם) אינם נמחקים. כל הטוקנים נשארים תקפים.

שלב שיחה: כשאורח לוחץ על הקישור, שלוש שאילתות רצות ברצף: Invite.findOne({ token }) →

Guest.findById(guestId) → Campaign.findById(campaignId). ה-Session מאותחל מנתוני Guest ו-Campaign. Campaign.find patch-בכתבים.

שלב קריאה: GET /api/campaigns/:id שולף Campaign אחד ואת כל ה-Guests שלו. סטטיסטיקות (count, YES response rate, total headcount) מחושבות בצד הלקוח מהרשימה - ללא aggregation pipeline. זה מקובל לגדלי קמפיין רגילים, לקמפיינים גדולים ניתן להוסיף אינדקס מורכב { campaignId, rsvpStatus } ו-group aggregation.

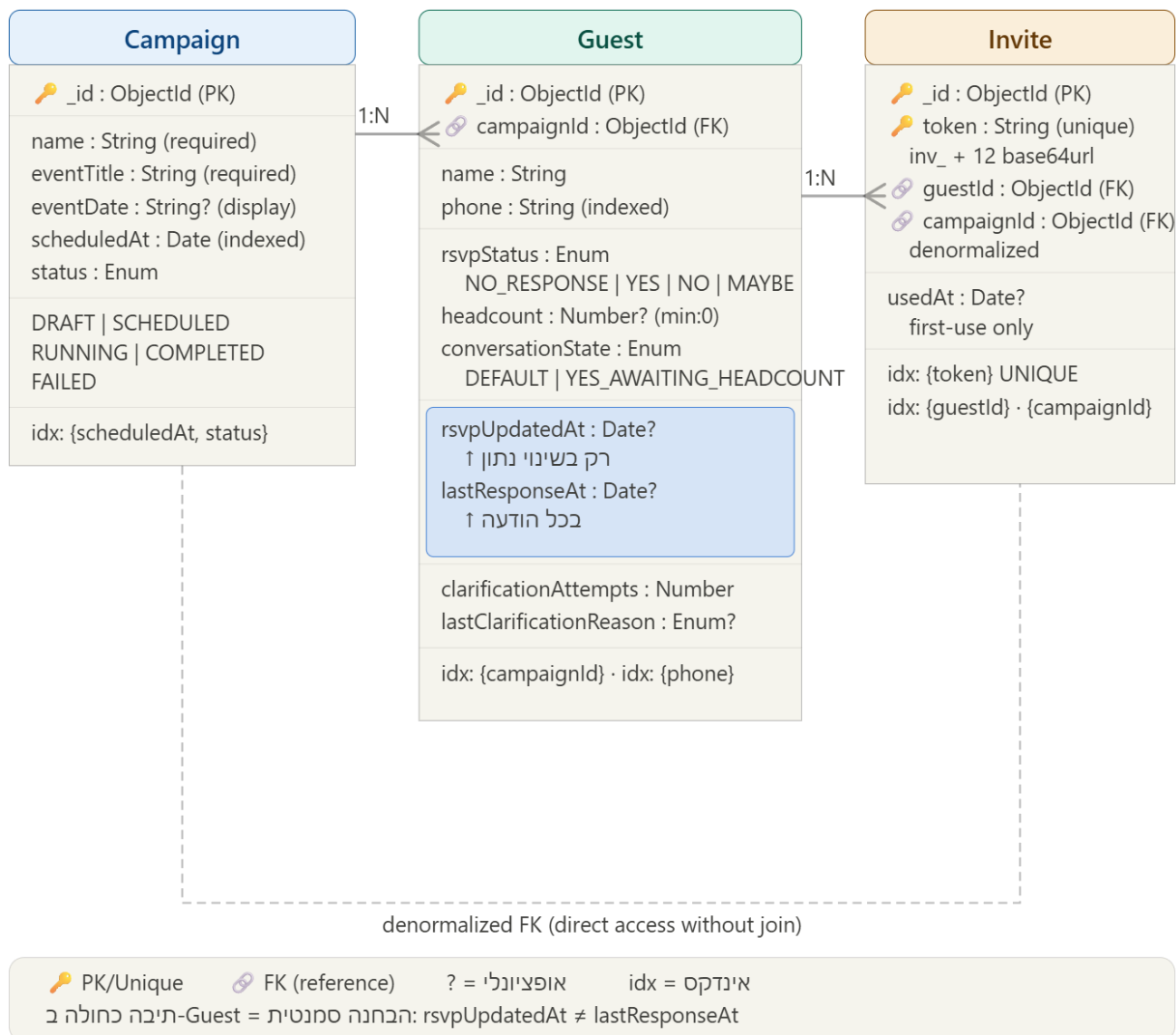
7.4 שיקולי עיצוב

בתכנון המערכת הושם דגש על שילוב בין תשתיות תוכנה יציבות ומוכחות לבין כלי בינה מלאכותית מתקדמים.

כבר בשלב הצעת המחקר, הוחלט להתבסס על Stack טכנולוגי המהווה סטנדרט בתעשיית ההייטק. בחירה זו נבעה מתוך היכרות מעמיקה וניסיון עשיר של כותב העבודה עם כלים אלו כמפתח.

- **MongoDB:** נבחר כבסיס הנתונים בשל הגמישות של מודל ה-Document-based, המאפשר לאחסן היסטוריית שיחות משתנה ומבני נתונים של RSVP ללא צורך בסכמה קשיחה שעלולה להגביל את התפתחות הסוכן.

איור 13 - תרשים ERD - סכמת בסיס הנתונים



תרשים מודל בסיס הנתונים של EZ-Event-BOT. שלוש האוספים (collections) עם קשרי 1:N. תיבת ה-Guest מסמנת בכחול את שני שדות הזמן השונים סמנטית. הקו המקווקו מ-Campaign ל-Invite מסמן את ה-denormalization המכוון. אינדקס { token: 1 } על Invite הוא הנקודה הקריטית לביצועים, כל כניסה לשיחה תלויה בו.